

Unidade IV

7 OBSERVABILIDADE E RESILIÊNCIA

Uma aplicação web não se encerra no momento em que seu código é escrito ou em que suas funcionalidades respondem corretamente durante testes iniciais. Quando passa a ser executada em um ambiente real, ela fica sujeita a variações de carga, falhas de rede, indisponibilidade temporária de serviços externos, erros de configuração, dados inesperados, lentidão em consultas, exceções não previstas e comportamentos que nem sempre podem ser reproduzidos facilmente em desenvolvimento. Por isso, além de projetar funcionalidades, é necessário projetar a capacidade de compreender a aplicação em execução e de mantê-la operante diante de condições adversas. Observabilidade e resiliência são propriedades operacionais que respondem a essa necessidade.

Observabilidade é a capacidade de inferir o estado interno de um sistema a partir dos sinais que ele produz durante sua execução. Em aplicações ASP.NET Core, esses sinais podem incluir registros de eventos, métricas, rastreamentos distribuídos, informações sobre requisições, exceções, tempos de resposta, consumo de recursos e resultados de operações críticas. O objetivo não é apenas acumular dados técnicos, mas permitir que desenvolvedores e equipes de operação compreendam o que está acontecendo, identifiquem anomalias, investiguem causas e tomem decisões fundamentadas. Uma aplicação observável não é aquela que simplesmente registra muitas informações, mas aquela que produz sinais úteis, contextualizados e interpretáveis.

No PetMatch, a observabilidade possui relação direta com a confiança no serviço prestado. Se uma pessoa interessada em adoção envia um formulário e a operação falha, é necessário compreender se o problema ocorreu na validação, na persistência, na comunicação com outro serviço ou na infraestrutura. Se uma organização tenta cadastrar um pet e a página responde lentamente, é preciso identificar se a causa está no banco de dados, na quantidade de dados carregados, em uma dependência externa ou no próprio servidor. Se determinados usuários relatam dificuldade para acessar uma área protegida, a equipe precisa investigar o fluxo de autenticação, autorização e sessão. Sem sinais adequados, a aplicação torna-se opaca; com observabilidade, ela passa a oferecer evidências para diagnóstico.

Os registros de eventos, frequentemente chamados de logs, constituem uma das formas mais conhecidas de observabilidade. Eles permitem narrar acontecimentos relevantes da aplicação, como início e término de operações, falhas inesperadas, tentativas de acesso negado, exceções tratadas e mudanças importantes de estado. No entanto, logs eficazes exigem critério. Informações excessivas podem gerar ruído e dificultar a investigação; informações insuficientes podem tornar um problema invisível. Além disso, registros devem respeitar a privacidade e a segurança dos usuários, evitando exposição indevida de dados sensíveis. Em um sistema como o PetMatch, que pode lidar com contatos, perfis e manifestações de interesse, registrar bem significa equilibrar diagnóstico técnico e responsabilidade sobre os dados.

As métricas oferecem outra perspectiva. Enquanto logs descrevem eventos específicos, métricas permitem acompanhar tendências e quantificar comportamentos ao longo do tempo. Taxa de requisições, duração média de respostas, quantidade de erros, uso de memória, disponibilidade de endpoints e desempenho de operações de banco de dados são exemplos de sinais que ajudam a perceber a saúde geral da aplicação. Uma métrica isolada raramente conta toda a história, mas sua evolução pode revelar degradação progressiva, picos de uso, gargalos recorrentes ou efeitos de uma mudança recente. Para o PetMatch, métricas podem indicar, por exemplo, se uma funcionalidade de busca de pets está ficando lenta à medida que o volume de cadastros cresce.

O rastreamento, por sua vez, ajuda a acompanhar o percurso de uma requisição por meio de diferentes partes da aplicação e, em arquiteturas distribuídas, por diferentes serviços. Ele permite compreender como uma solicitação se desloca entre endpoint, regras de aplicação, acesso ao banco, chamadas externas e geração de resposta. Essa visão é importante quando um problema não está concentrado em um único ponto evidente. Uma página lenta pode ser consequência de uma consulta ineficiente, de uma chamada externa demorada ou de várias pequenas operações acumuladas. O rastreamento ajuda a reconstruir o caminho da requisição e a localizar onde o tempo foi gasto ou onde a falha ocorreu.

A resiliência, embora relacionada à operação, possui foco diferente. Ela expressa a capacidade da aplicação de continuar oferecendo comportamento adequado mesmo quando algo falha ou demora mais do que o esperado. Sistemas reais dependem de componentes que podem se tornar indisponíveis, responder lentamente ou produzir erros intermitentes. A resiliência não parte da ideia de que falhas são exceções improváveis, mas da compreensão de que falhas fazem parte do ambiente de execução. Uma aplicação resiliente procura limitar danos, evitar que uma falha localizada comprometa todo o sistema, oferecer respostas controladas e recuperar-se quando as condições voltam à normalidade.

No PetMatch, a resiliência pode ser percebida na forma como a aplicação lida com situações adversas sem abandonar o usuário diante de mensagens incompreensíveis ou estados inconsistentes. Se uma consulta temporariamente não puder recuperar a lista de pets disponíveis, a aplicação deve responder de modo controlado, preservando a navegação e comunicando a situação adequadamente. Se um serviço externo usado para complementar informações estiver indisponível, o sistema deve evitar que toda a experiência de adoção seja interrompida desnecessariamente. Se uma operação crítica falhar, é preciso impedir registros parciais que comprometam a integridade dos dados. Assim, resiliência envolve tanto decisões técnicas quanto cuidado com a continuidade da experiência.

Estratégias de resiliência podem incluir tratamento adequado de exceções, definição de tempos limite, repetição controlada de operações transitórias, isolamento de falhas, degradação graciosa e proteção contra sobrecarga. Cada estratégia precisa ser aplicada com discernimento. Repetir indefinidamente uma operação pode piorar a sobrecarga; ocultar todos os erros pode impedir diagnóstico; interromper agressivamente uma funcionalidade pode prejudicar o usuário. O projeto resiliente exige compreender a natureza da operação, sua criticidade, sua dependência de outros componentes e o impacto de sua falha. Em aplicações web, resiliência é uma parte planejada da arquitetura operacional.



Observação

Observabilidade e resiliência se fortalecem mutuamente. Não é possível tornar uma aplicação resiliente de forma madura sem compreender como ela falha, onde degrada, quais dependências são mais sensíveis e quais operações exigem maior proteção. Ao mesmo tempo, a observabilidade ganha valor quando seus sinais orientam respostas práticas, ajustes arquiteturais e mecanismos de recuperação. Uma aplicação que apenas registra falhas, mas não possui meios de contê-las, permanece frágil. Uma aplicação que tenta se recuperar sem produzir evidências de seu comportamento torna-se difícil de manter. O equilíbrio entre ver, diagnosticar, proteger e recuperar define a qualidade operacional do sistema.

Este tópico introduz observabilidade e resiliência como dimensões indispensáveis ao desenvolvimento web profissional com .NET. Depois de compreender requisições, páginas, persistência, APIs, integração cliente-servidor e segurança, é necessário observar a aplicação em funcionamento e prepará-la para o mundo real, no qual falhas e variações são inevitáveis. No percurso evolutivo do PetMatch, essa etapa amplia a visão do leitor para além da implementação funcional, mostrando que um sistema responsável precisa ser compreensível quando executa, diagnosticável quando falha e robusto quando encontra condições adversas. Desenvolver bem não significa apenas fazer a aplicação funcionar, mas torná-la capaz de sustentar seu funcionamento com clareza, controle e confiança.

7.1 Logging, métricas, tracing e Health Checks

Como vimos, a observabilidade é a capacidade de compreender o comportamento de uma aplicação enquanto ela está em execução. Uma aplicação web pode exibir páginas corretamente no navegador e, ainda assim, ser difícil de analisar quando algo falha, quando uma operação demora, quando o banco de dados fica inacessível ou quando um usuário encontra uma restrição de acesso. Em sistemas web modernos, a execução envolve requisições HTTP, autenticação, autorização, persistência, consultas, redirecionamentos, validações e respostas de erro.



Lembrete

Para interpretar esse conjunto, o desenvolvedor precisa de sinais internos que expliquem o que aconteceu, onde aconteceu e em que condição a aplicação se encontrava naquele momento.

O logging é o registro de eventos relevantes produzidos pela aplicação. No ASP.NET Core, logs são integrados à infraestrutura da plataforma e podem ser emitidos por controllers, endpoints, serviços e pelo próprio pipeline HTTP. Um log de nível `Information` descreve uma operação esperada, como a listagem de pets. Um log de nível `warning` indica uma situação que merece atenção, como uma

tentativa de login inválida. Um log de nível `Error` registra uma falha que impediu a conclusão de uma operação. O valor do logging está no contexto: a mensagem deve informar o evento, seus dados operacionais relevantes e o ponto da aplicação em que ocorreu.

Logs estruturados são preferíveis a mensagens montadas manualmente porque preservam os valores como propriedades nomeadas. Em vez de concatenar textos, o código registra campos como página consultada, tamanho da página, identificador do pet ou resultado da operação. Esse formato melhora a leitura no desenvolvimento e prepara a aplicação para coleta futura por ferramentas especializadas. Em fluxos de autenticação, o cuidado é ainda maior: a aplicação pode registrar que uma tentativa de login falhou, mas não deve registrar senha, cookie, token ou qualquer segredo. No PetMatch, essa regra orienta os registros em `AuthEndpoints.cs`, `AccountController.cs` e nos fluxos protegidos de cadastro e edição.

As métricas representam medições agregadas do comportamento da aplicação. Enquanto um log descreve um evento específico, uma métrica mostra volume, frequência ou tendência. Um contador pode indicar quantas vezes a listagem de pets foi consultada, quantas páginas de detalhe foram acessadas ou quantas tentativas de login foram inválidas. Essa diferença é fundamental: logs ajudam a investigar eventos e métricas ajudam a perceber padrões. No .NET, `System.Diagnostics.Metrics` permite criar instrumentos como contadores sem adicionar pacotes externos, o que torna o recurso adequado para uma introdução didática e controlada.

Já o tracing acompanha o percurso de uma operação. Um trace permite demarcar uma atividade e relacioná-la ao fluxo de execução que está ocorrendo. Em uma requisição de listagem, por exemplo, a atividade pode envolver a chegada ao endpoint, a aplicação de filtros, a consulta ao banco e a geração da resposta. No .NET, `ActivitySource` é o mecanismo usado para criar essas atividades. Mesmo sem exportar traces para uma ferramenta externa, organizar o código com `ActivitySource` ensina uma prática importante: operações relevantes devem ser nomeadas e observáveis.

As Health Checks são verificações objetivas de saúde da aplicação. Uma verificação de vida responde se o processo está ativo e consegue responder. Uma verificação de prontidão responde se a aplicação está em condição de atender adequadamente, considerando dependências essenciais. No PetMatch, essa distinção aparece em dois endpoints: `/health/live`, que confirma que a aplicação está viva (figura 49), e `/health/ready`, que verifica também a prontidão do SQLite por meio do `PetMatchDbContext` (figura 50). Assim, o leitor consegue observar no navegador uma resposta simples e direta sobre a condição operacional da aplicação.

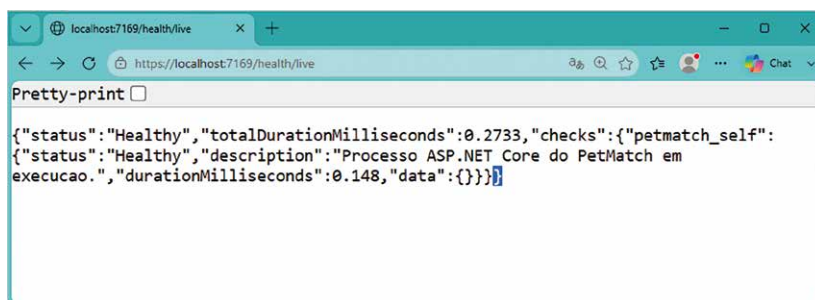


Figura 49 – Verificação de vida em `/health/live`

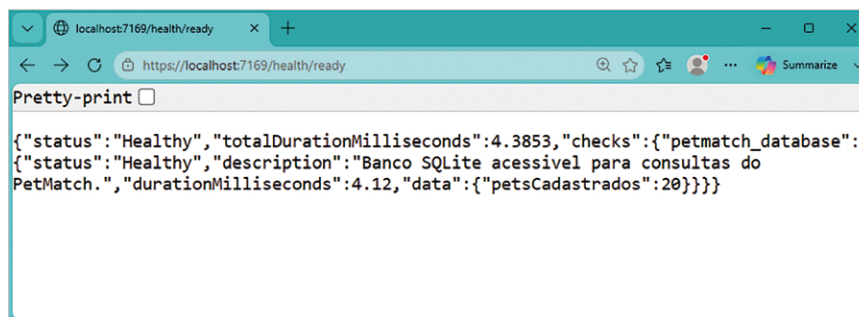


Figura 50 – Verificação de prontidão em /health/ready

Nesta fase, o PetMatch volta a uma execução unificada centrada em `PetMatch.Web`. O projeto React criado nos tópicos de front-end permanece preservado fisicamente no repositório como parte da evolução do estudo, mas foi removido da solução ativa, do build e da inicialização. A prática passa a ocorrer diretamente no ASP.NET Core, com MVC, Minimal APIs, Identity, SQLite, logs, métricas, tracing e Health Checks no mesmo projeto. Ao abrir a aplicação, a rota / redireciona para /Pets, exibindo a área MVC com cards de pets, navegação, login, cadastro e páginas protegidas.

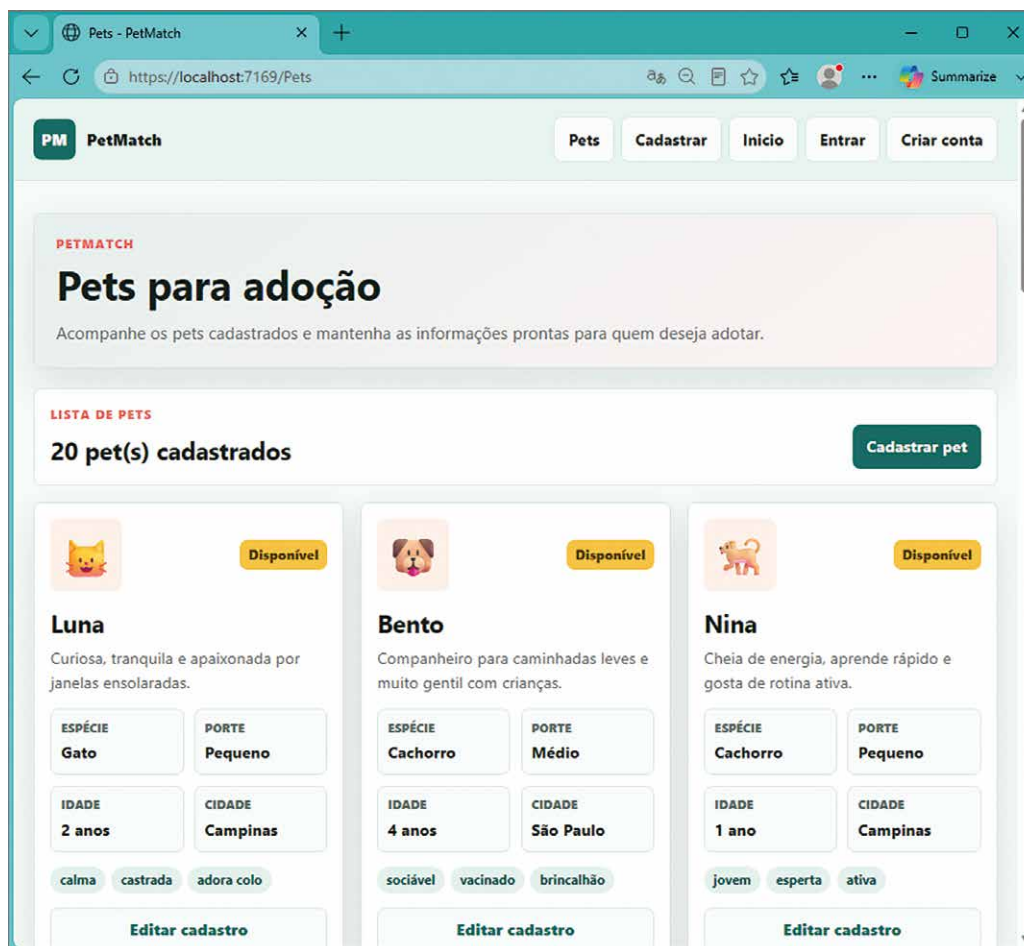


Figura 51 – Nova página /Pets sem o React

Para organizar o tracing, foi criada a pasta `Observability` no projeto `PetMatch.Web`. O arquivo `PetMatch.Web/Observability/PetMatchDiagnostics.cs` concentra o `ActivitySource` usado pela aplicação. O papel desse código é fornecer um ponto único para nomear a origem das atividades do `PetMatch`.

```
using System.Diagnostics;

namespace PetMatch.Web.Observability;

public static class PetMatchDiagnostics
{
    public const string ActivitySourceName = "PetMatch.Web";

    public static readonly ActivitySource ActivitySource =
new(ActivitySourceName);
}
```

Esse trecho cria uma origem de tracing chamada `PetMatch.Web`. Endpoints e controllers podem iniciar atividades a partir dela, mantendo consistência nos nomes e evitando strings repetidas em vários arquivos. O efeito prático é deixar o código preparado para acompanhar operações importantes, como consultas de pets e fluxos de autenticação.

As métricas ficam centralizadas em `PetMatch.Web/Observability/PetMatchMetrics.cs`. Esse arquivo reúne contadores nativos do .NET relacionados às operações mais importantes do `PetMatch`.

```
using System.Diagnostics.Metrics;

namespace PetMatch.Web.Observability;

public static class PetMatchMetrics
{
    public const string MeterName = "PetMatch.Web";

    private static readonly Meter Meter = new(MeterName);

    public static readonly Counter<long> PetListRequests =
        Meter.CreateCounter<long>("petmatch.pet_list_requests");

    public static readonly Counter<long> PetDetailsRequests =
        Meter.CreateCounter<long>("petmatch.pet_details_requests");

    public static readonly Counter<long> SuccessfulLogins =
        Meter.CreateCounter<long>("petmatch.successful_logins");

    public static readonly Counter<long> InvalidLogins =
        Meter.CreateCounter<long>("petmatch.invalid_logins");
}
```

Esse código define um `Meter` e quatro contadores. A aplicação pode incrementar esses contadores quando a listagem de pets é acessada, quando um detalhe é consultado, quando um login é bem-sucedido

ou quando uma tentativa inválida ocorre. O navegador continua exibindo as mesmas páginas, mas a aplicação passa a produzir sinais internos sobre seu uso.

A verificação de prontidão do banco foi criada em `PetMatch.Web/HealthChecks/PetMatchDatabaseHealthCheck.cs`. Esse Health Check usa o `PetMatchDbContext` para consultar o SQLite e confirmar que a persistência está acessível.

```
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Diagnostics.HealthChecks;
using PetMatch.Web.Data;

namespace PetMatch.Web.HealthChecks;

public sealed class PetMatchDatabaseHealthCheck(PetMatchDbContext dbContext)
    : IHealthCheck
{
    public async Task<HealthCheckResult> CheckHealthAsync(
        HealthCheckContext context,
        CancellationToken cancellationToken = default)
    {
        try
        {
            var petsCadastrados = await
dbContext.Pets.CountAsync(cancellationToken);

            return HealthCheckResult.Healthy(
                "Banco de dados acessível.",
                new Dictionary<string, object>
                {
                    ["petsCadastrados"] = petsCadastrados
                });
        }
        catch (Exception exception)
        {
            return HealthCheckResult.Unhealthy(
                "Não foi possível consultar o banco de dados.",
                exception);
        }
    }
}
```

Esse código executa uma consulta mínima à tabela de pets. Quando a consulta funciona, o endpoint de prontidão pode informar que o banco está acessível e incluir a quantidade de pets cadastrados. Quando ocorre exceção, o Health Check retorna estado não saudável, permitindo identificar rapidamente que a aplicação não está pronta para operar corretamente.

O registro dos Health Checks e o mapeamento dos endpoints ficam em `PetMatch.Web/Program.cs`. O trecho a seguir mostra a configuração essencial.

```
builder.Services
    .AddHealthChecks()
    .AddCheck<PetMatchDatabaseHealthCheck>("sqlite");

app.MapHealthChecks("/health/live", new HealthCheckOptions
{
```



```
Predicate = _ => false
});

app.MapHealthChecks("/health/ready");
```

Com essa configuração, /health/live responde à verificação de vida da aplicação, ao passo que /health/ready executa as verificações registradas, incluindo o acesso ao SQLite. No navegador, /health/live retorna Healthy; /health/ready retorna Healthy e inclui dados como petsCadastrados.

Os endpoints de pets também passam a emitir sinais de observabilidade. Em PetMatch.Web/Endpoints/PetEndpoints.cs, a consulta de listagem pode iniciar uma atividade, registrar um log estruturado e incrementar uma métrica.

```
using var activity = PetMatchDiagnostics.ActivitySource.StartActivity("Listar
pets");

logger.LogInformation(
    "Listagem de pets solicitada. Página: {Page}; Tamanho: {PageSize}",
    query.Page,
    query.PageSize);

PetMatchMetrics.PetListRequests.Add(1);
```

Esse trecho demonstra a combinação dos três sinais. A atividade representa o fluxo, o log descreve o evento com contexto e a métrica registra a contagem agregada da operação. A resposta da API continua sendo o envelope paginado já implementado, mas o comportamento interno passa a ser observável. A figura 52 mostra um exemplo de mensagem no Output do Visual Studio, a partir de uma ação feita na interface do navegador.

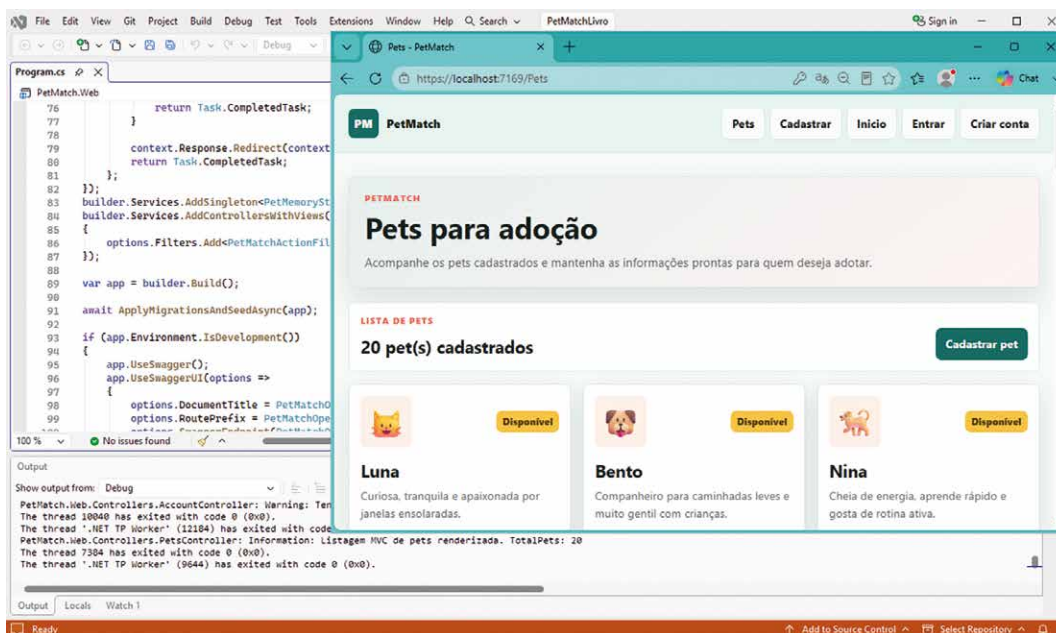


Figura 52 – Na janela de Output do Visual Studio (canto inferior esquerdo) aparece a informação "Listagem MVC de pets renderizada" e o número total de pets

Nos fluxos de autenticação, `PetMatch.Web/Endpoints/AuthEndpoints.cs` e `PetMatch.Web/Controllers/AccountController.cs` registram eventos sem expor dados sensíveis. A tentativa inválida de login (figura 53), por exemplo, pode ser registrada com nível de atenção e incrementar uma métrica.

```
logger.LogWarning(  
    "Tentativa de login inválida para o e-mail {Email}.",  
    request.Email);  
  
PetMatchMetrics.InvalidLogins.Add(1);
```

Esse código informa o evento e preserva a segurança. O log ajuda a compreender o comportamento da aplicação (figura 54) e a métrica permite contar tentativas inválidas, mas nenhum segredo é gravado. Esse padrão é essencial em qualquer aplicação com autenticação.

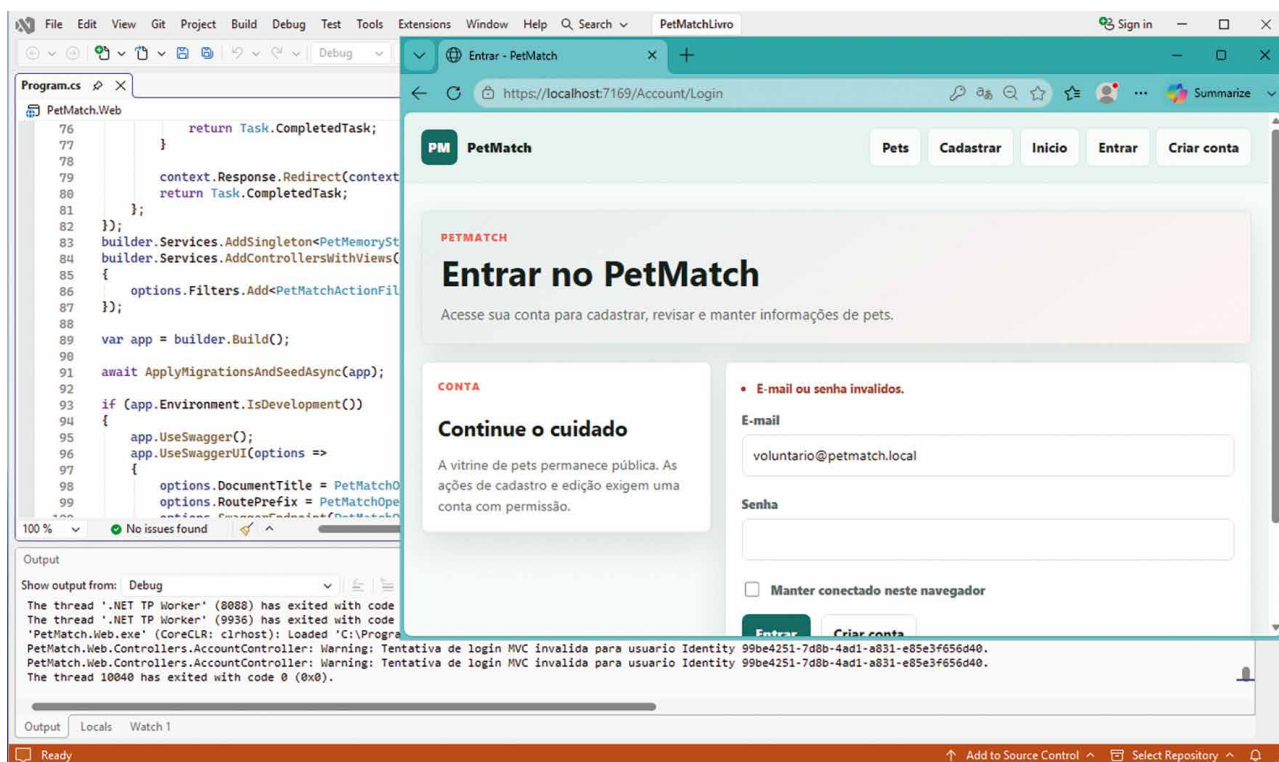


Figura 53 – Na janela de Output do Visual Studio (canto inferior esquerdo) aparece o alerta da tentativa de login inválida

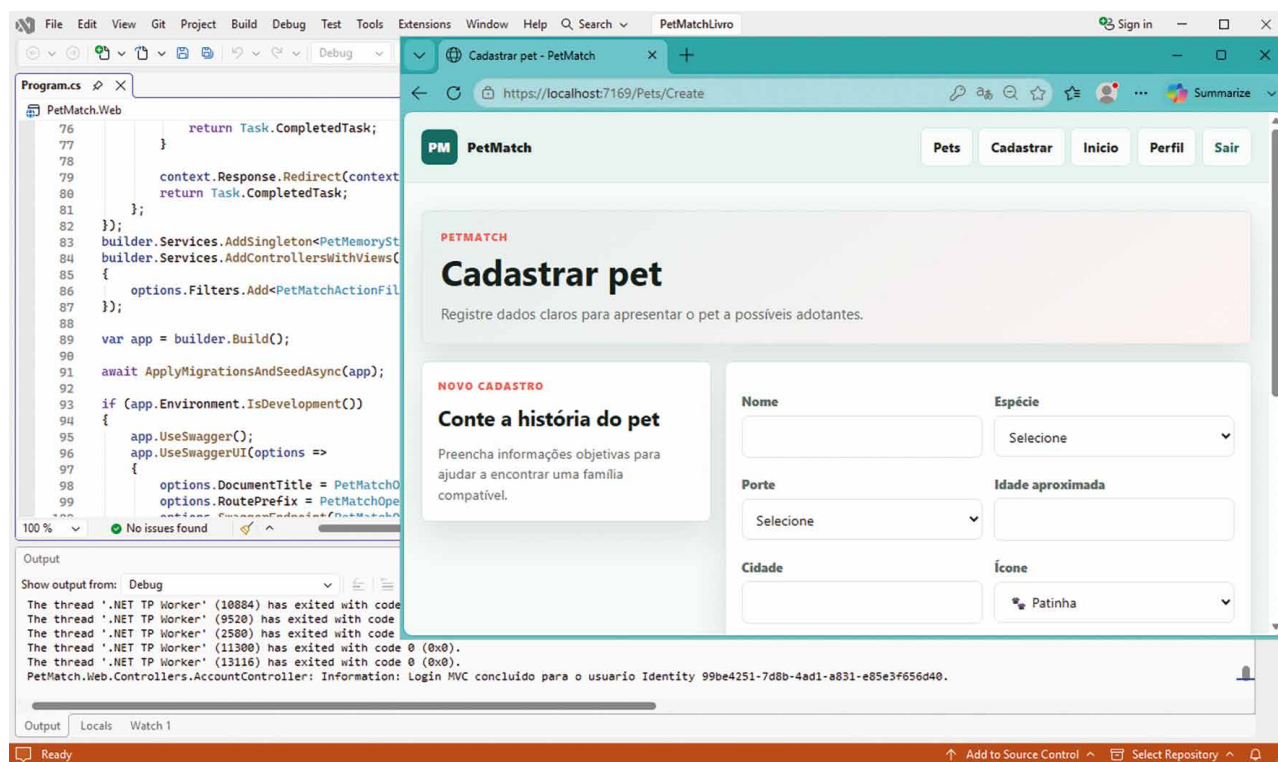


Figura 54 – Na janela de Output do Visual Studio (canto inferior esquerdo) aparece a informação da tentativa de login bem-sucedida

Nos fluxos MVC, `PetMatch.Web/Controllers/PetsController.cs` registra eventos de cadastro e edição. Esses logs acompanham ações reais da interface do usuário, como adicionar um novo pet ou atualizar seu cadastro.

```
_logger.LogInformation(
    "Pet cadastrado com sucesso. PetId: {PetId}; Nome: {Nome}",
    pet.Id,
    pet.Nome);
```

Esse registro descreve uma operação de negócio sem incluir dados sigilosos. Ao cadastrar um pet pela interface MVC (figuras 55 e 56), o leitor pode observar no Output do Visual Studio que a operação foi concluída, qual registro foi afetado e em que ponto do fluxo ela ocorreu.

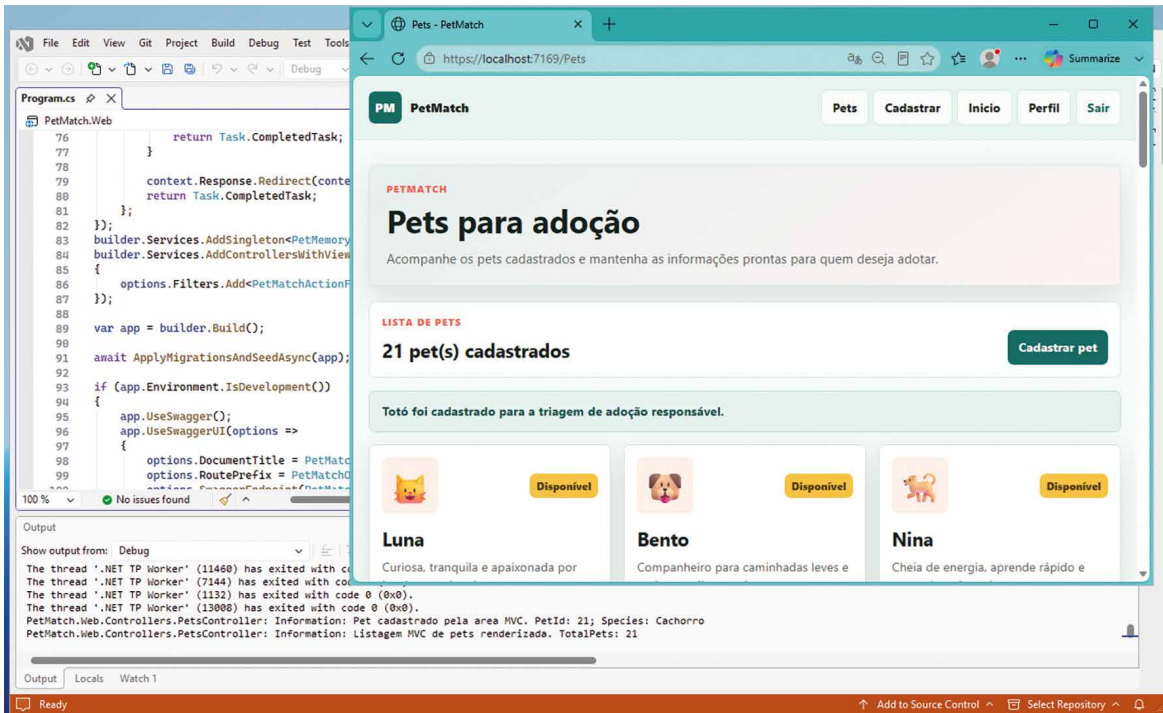


Figura 55 – Na janela de Output do Visual Studio (canto inferior esquerdo) aparece a informação da tentativa de cadastro bem-sucedida do pet

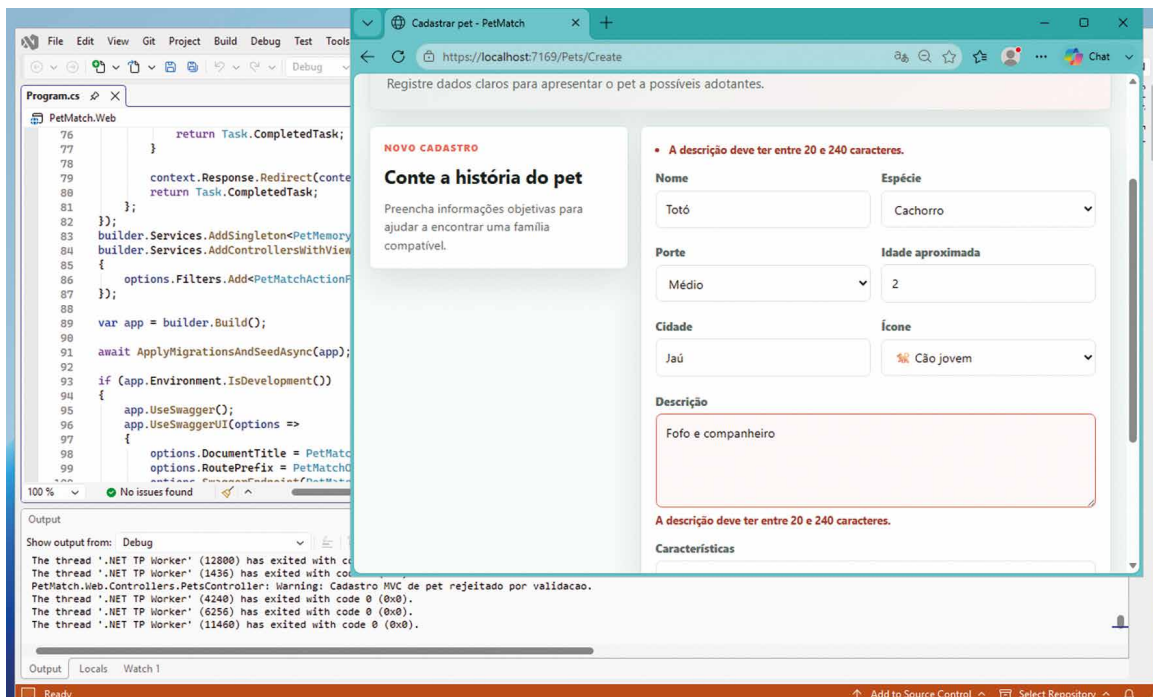


Figura 56 – Na janela de Output do Visual Studio (canto inferior esquerdo) aparece o alerta da tentativa de cadastro mal-sucedida do pet: rejeitado por validação

Os níveis de log são configurados nos arquivos `PetMatch.Web/appsettings.json` e `PetMatch.Web/appsettings.Development.json`. Em desenvolvimento, é útil manter os logs da aplicação visíveis e reduzir ruídos de infraestrutura.

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning",
      "Microsoft.EntityFrameworkCore.Database.Command": "Warning",
      "PetMatch.Web": "Information"
    }
  }
}
```

Essa configuração orienta o ASP.NET Core sobre quais níveis exibir. O resultado é uma janela Output mais útil no Visual Studio 2026, com eventos da aplicação aparecendo sem excesso de mensagens internas do framework.

A implementação deste subtópico criou `PetMatchDiagnostics`, `PetMatchMetrics` e `PetMatchDatabaseHealthCheck`, além das pastas `Observability` e `HealthChecks` dentro de `src/PetMatch.Web`. Também foram ajustados `Program.cs`, `PetEndpoints.cs`, `AuthEndpoints.cs`, `PetsController.cs`, `AccountController.cs`, `PetMatch.Web.csproj`, `launchSettings.json`, `appsettings.json`, `appsettings.Development.json`, `PetMatchLivro.sln` e `PetMatchLivro.slnLaunch.user`.

O resultado esperado no navegador é a aplicação abrindo em `/Pets`, com a listagem MVC de vinte pets, CSS aplicado, navegação, login, cadastro e páginas protegidas funcionando; os endpoints `/health/live` e `/health/ready` exibem JSON de saúde no navegador.

7.2 Políticas de resiliência (retry/circuit breaker) e rate limiting/output caching (quando disponíveis)

A estabilidade operacional é a capacidade de uma aplicação manter comportamento previsível quando enfrenta falhas transitórias, excesso de requisições ou operações repetidas que podem ser atendidas com menor custo. Em aplicações web, a instabilidade nem sempre nasce de um erro de programação. Ela pode surgir de uma dependência lenta, de uma chamada HTTP temporariamente indisponível, de um pico de acessos, de uma consulta pública repetida muitas vezes ou de tentativas sucessivas contra endpoints sensíveis. Políticas de resiliência, limitação de requisições e cache de saída existem para transformar essas situações em comportamentos explícitos, configuráveis e observáveis.

O `retry` é a repetição controlada de uma operação que falhou por uma condição transitória. A palavra "controlada" é decisiva: uma política de `retry` precisa definir quantidade máxima de tentativas, intervalo entre elas e tipo de falha que justifica nova tentativa. Esse mecanismo é adequado para operações de leitura ou chamadas idempotentes, nas quais repetir a operação não provoca efeito duplicado. Em contrapartida, operações como cadastro, edição, login ou qualquer ação com efeito de gravação exigem

mais cautela, pois a repetição automática pode produzir resultados indesejados. No PetMatch, o retry foi usado em uma chamada de leitura para obter uma dica de adoção responsável, preservando operações sensíveis fora desse comportamento.

O circuit breaker é uma política de proteção aplicada quando uma dependência começa a falhar repetidamente. Se a aplicação insiste em chamar um recurso que já demonstrou instabilidade, ela consome conexões, threads, tempo de resposta e capacidade de processamento. O circuit breaker observa falhas sucessivas e, ao atingir determinado limite, abre o circuito por um período. Enquanto o circuito está aberto, a aplicação evita novas chamadas e usa uma resposta alternativa. Depois, permite uma nova tentativa. A finalidade não é corrigir a dependência, mas impedir que a aplicação agrave o problema insistindo em uma chamada que provavelmente falhará.

O rate limiting é a limitação do volume de requisições aceitas em determinado intervalo. Ele protege a aplicação contra uso excessivo, automações agressivas, erros de cliente e abuso de endpoints sensíveis. Uma API pública de leitura pode ter limites mais generosos, enquanto login e cadastro devem receber limites mais restritivos. Quando o limite é excedido, o status HTTP adequado é 429 Too Many Requests, indicando que o cliente fez chamadas demais em curto período. No PetMatch, foram criadas políticas para endpoints públicos e autenticação, preservando a experiência normal do usuário e adicionando proteção contra excesso de chamadas.

Já o output caching é o armazenamento temporário da resposta produzida pela aplicação. Quando uma resposta é pública, segura e independente do usuário atual, ela pode ser reaproveitada por um curto período, reduzindo consultas, processamento e tempo de resposta. O cuidado principal está no escopo: páginas de perfil, login, áreas autenticadas ou respostas personalizadas por cookie, claims ou sessão não devem ser armazenadas como cache compartilhado. No PetMatch, o output caching foi aplicado a respostas públicas: GET /api/pets, GET /api/pets/{id:int} e dicas públicas de adoção. As páginas dependentes de autenticação permanecem fora do cache.



Lembrete

O PetMatch permanece em uma fase unificada centrada em PetMatch.Web.

A aplicação executada no Visual Studio 2026 é a área ASP.NET Core com MVC, Minimal APIs, Identity, SQLite, observabilidade e políticas de estabilidade operacional. A tela principal continua sendo /Pets, com cards de pets, navegação, login, cadastro e páginas protegidas. O resultado deste subtópico não é um painel técnico, mas uma aplicação mais resistente: a listagem pública continua agradável para o usuário, os endpoints públicos passam a ter cache e limites, e a dica de cuidado responsável usa retry e circuit breaker com fallback amigável.



Observação

"Fallback amigável" é uma resposta alternativa, clara e segura para o usuário quando uma funcionalidade falha temporariamente. A título de exemplo, imagine uma situação de falha que em vez de aparecer um erro técnico tal como HTTP 500, Timeout, Service unavailable ou Exception, a aplicação mostra uma mensagem compreensível e útil para o usuário: "No momento não conseguimos carregar uma dica personalizada. Enquanto isso, lembre-se de manter água fresca disponível e consultar um veterinário em caso de dúvida".

Para centralizar os nomes das políticas, foi criada a pasta `Resilience` no projeto `PetMatch.Web`. O arquivo `PetMatch.Web/Resilience/PetMatchResiliencePolicies.cs` reúne constantes usadas por rate limiting, output caching e resiliência. Antes de aplicar políticas em vários pontos da aplicação, é útil dar nomes claros a elas, pois esses nomes passam a fazer parte da configuração operacional do sistema.

```
namespace PetMatch.Web.Resilience;

public static class PetMatchResiliencePolicies
{
    public const string PublicApiRateLimit = "petmatch-public-api";
    public const string AuthRateLimit = "petmatch-auth";
    public const string PublicPetsCache = "petmatch-public-pets";
    public const string AdoptionTipsCache = "petmatch-adoption-tips";
}
```

Esse código evita strings soltas em `Program.cs`, endpoints e controllers. A aplicação passa a ter nomes consistentes para políticas públicas, políticas de autenticação e políticas de cache. O efeito prático é tornar explícito qual comportamento está sendo aplicado a cada endpoint.

A configuração central das políticas fica em `PetMatch.Web/Program.cs`. O trecho a seguir representa o registro de rate limiting e output caching, usando recursos nativos do ASP.NET Core.

```
builder.Services.AddRateLimiter(options =>
{
    options.AddFixedWindowLimiter(PetMatchResiliencePolicies.PublicApiRateLimit,
limiterOptions =>
    {
        limiterOptions.PermitLimit = 6;
        limiterOptions.Window = TimeSpan.FromSeconds(30);
    });

    options.AddFixedWindowLimiter(PetMatchResiliencePolicies.AuthRateLimit,
limiterOptions =>
    {
```



```
        limiterOptions.PermitLimit = 4;
        limiterOptions.Window = TimeSpan.FromMinutes(1);
    });
});

builder.Services.AddOutputCache(options =>
{
    options.AddPolicy(PetMatchResiliencePolicies.PublicPetsCache, builder =>
        builder.Expire(TimeSpan.FromSeconds(20)));

    options.AddPolicy(PetMatchResiliencePolicies.AdoptionTipsCache, builder =>
        builder.Expire(TimeSpan.FromSeconds(30)));
});
```

Esse código cria dois limites de requisição e duas políticas de cache. Os valores são curtos para fins didáticos, permitindo que o leitor observe o comportamento no navegador e no Output do Visual Studio. A política pública aceita chamadas de leitura em volume moderado e a política de autenticação é mais restritiva. O cache de pets e dicas dura poucos segundos, reduzindo repetição de processamento sem mascarar mudanças por muito tempo.

Depois de registrar os serviços, os middlewares precisam entrar no pipeline da aplicação. Ainda em `PetMatch.Web/Program.cs`, as chamadas a seguir ativam os recursos durante o processamento das requisições.

```
app.UseRateLimiter();

app.UseOutputCache();
```

Essas chamadas fazem com que as políticas aplicadas aos endpoints tenham efeito real. Sem elas, as configurações existiriam nos serviços, mas não influenciariam as respostas HTTP. A aplicação continua preservando MVC, Minimal APIs, autenticação, autorização, Health Checks e observabilidade.

A demonstração de retry e circuit breaker foi implementada de forma didática, sem `Microsoft.Extensions.Http.Resilience`. O arquivo `PetMatch.Web/Services/AdoptionTipClient.cs` concentra a chamada de leitura que busca uma dica de adoção responsável. O cliente usa tentativas controladas, registra falhas transitórias e recorre a um estado de circuito representado por `AdoptionTipCircuitState`.

```
public async Task<AdoptionTipResponse> GetTipAsync(CancellationToken
cancellationToken)
{
    if (_circuitState.IsOpen)
    {
        return AdoptionTipResponse.Fallback();
    }

    for (var attempt = 1; attempt <= 3; attempt++)
    {
        try
        {
```

```

        var response = await _httpClient.GetFromJsonAsync<AdoptionTipResponse>(
            "/api/dicas/adocao-parceiro",
            cancellationToken);

        _circuitState.ReportSuccess();

        return response ?? AdoptionTipResponse.Fallback();
    }
    catch (Exception exception) when (attempt < 3)
    {
        _logger.LogWarning(
            exception,
            "Falha transitória ao buscar dica de adoção. Tentativa {Attempt}.",
            attempt);
    }

    _circuitState.ReportFailure();

    return AdoptionTipResponse.Fallback();
}

```

Esse trecho mostra uma operação de leitura com comportamento resiliente. A aplicação tenta obter a dica, registra falhas temporárias, informa sucesso ou falha ao circuito e devolve uma resposta alternativa quando necessário. O usuário não vê uma tela quebrada; ele recebe uma orientação útil mesmo quando a dependência da dica não responde como esperado. A resposta usada pela dica foi criada em `PetMatch.Web/Services/AdoptionTipResponse.cs`. O tipo é pequeno porque representa conteúdo de apresentação, não um modelo de domínio complexo.

```

namespace PetMatch.Web.Services;

public sealed record AdoptionTipResponse(
    string Titulo,
    string Mensagem)
{
    public static AdoptionTipResponse Fallback() =>
        new(
            "Cuidado responsável",
            "Antes da adoção, confirme se sua rotina comporta tempo, espaço e acompanhamento veterinário.");
}

```

Esse código cria uma resposta com título e mensagem e define uma alternativa segura. A consequência prática é que a view MVC pode sempre exibir uma dica coerente, mesmo quando o cliente resiliente precisa acionar fallback. O endpoint público da dica foi criado em `PetMatch.Web/Endpoints/AdoptionTipEndpoints.cs`. Ele publica GET `/api/dicas/adocao-parceiro`, retornando uma orientação em JSON.

```

app.MapGet("/api/dicas/adocao-parceiro", () =>
    Results.Ok(new AdoptionTipResponse(

```

```
"Adaptação gradual",  
"Nos primeiros dias, mantenha água, descanso e uma rotina tranquila para  
o pet se sentir seguro.")))  
.CacheOutput(PetMatchResiliencePolicies.AdoptionTipsCache)  
.RequireRateLimiting(PetMatchResiliencePolicies.PublicApiRateLimit);
```

Esse endpoint é público, cacheável e limitado por política de requisições. Ele não cria uma tela técnica para o usuário final; apenas fornece conteúdo que a página /Pets pode apresentar como parte da experiência de adoção responsável.

Os endpoints públicos de pets foram ajustados em `PetMatch.Web/Endpoints/PetEndpoints.cs`. A listagem e o detalhe continuam entregando os contratos já conhecidos, mas agora recebem cache de saída e rate limiting.

```
group.MapGet("/", ListPetsAsync)  
.CacheOutput(PetMatchResiliencePolicies.PublicPetsCache)  
.RequireRateLimiting(PetMatchResiliencePolicies.PublicApiRateLimit);  
  
group.MapGet("/{id:int}", GetPetByIdAsync)  
.CacheOutput(PetMatchResiliencePolicies.PublicPetsCache)  
.RequireRateLimiting(PetMatchResiliencePolicies.PublicApiRateLimit);
```

Esse código aplica as políticas apenas a leituras públicas. A API continua retornando envelope paginado em GET /api/pets e Problem Details quando um pet não existe. A diferença é operacional: chamadas repetidas podem ser atendidas pelo cache, e chamadas excessivas podem receber 429 Too Many Requests (figura 57).

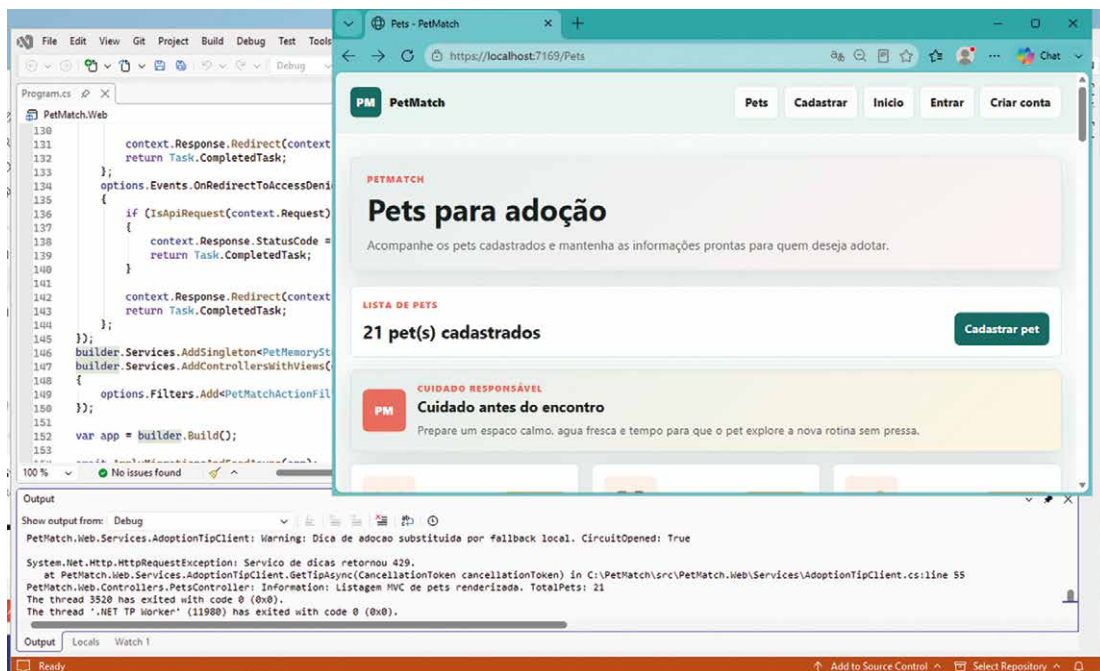


Figura 57 – Na janela de Output do Visual Studio (canto inferior esquerdo) aparece o alerta de circuito aberto e o 429

Os endpoints de autenticação foram ajustados em `PetMatch.Web/Endpoints/AuthEndpoints.cs`. Login e cadastro recebem uma política mais restritiva, pois são fluxos sensíveis.

```
auth.MapPost("/login", LoginAsync)
    .RequireRateLimiting(PetMatchResiliencePolicies.AuthRateLimit);

auth.MapPost("/register", RegisterAsync)
    .RequireRateLimiting(PetMatchResiliencePolicies.AuthRateLimit);
```

Esse trecho protege endpoints de autenticação contra excesso de tentativas. A medida não substitui validação, hash de senha, cookies seguros ou autorização, mas adiciona uma camada importante de proteção operacional.

A experiência visual aparece em `/Pets`. O arquivo `PetMatch.Web/Controllers/PetsController.cs` consome o `AdoptionTipClient` e envia a dica para a view. A página `PetMatch.Web/Views/Pets/Index.cshtml` exibe uma faixa "cuidado responsável" junto à listagem.

```
var adoptionTip = await _adoptionTipClient.GetTipAsync(cancellationToken);
ViewData["AdoptionTipTitle"] = adoptionTip.Titulo;
ViewData["AdoptionTipMessage"] = adoptionTip.Mensagem;
```

Esse código conecta resiliência a uma experiência real do usuário. A dica pode vir do endpoint público, ser reaproveitada por cache, passar por retry em falhas transitórias ou cair no fallback. Para o usuário, o resultado é uma orientação útil dentro da própria listagem de pets. A aparência da faixa foi definida em `PetMatch.Web/wwwroot/css/petmatch-mvc.css`. O objetivo é integrar o recurso à identidade visual da aplicação, sem aparência de painel técnico.

```
.responsible-care {
    margin: 1.5rem 0;
    padding: 1.25rem;
    border-radius: 1rem;
    background: #fff7ed;
    border: 1px solid #fed7aa;
}

.responsible-care strong {
    display: block;
    color: #9a3412;
    margin-bottom: .35rem;
}
```

Esse estilo cria uma área discreta, legível e coerente com a tela MVC. O resultado esperado em `/Pets` é a listagem com cards de pets e uma faixa de cuidado responsável, sem painel de diagnóstico ou tela administrativa.

A implementação criou `PetMatchResiliencePolicies`, `AdoptionTipClient`, `AdoptionTipCircuitState`, `AdoptionTipResponse` e `AdoptionTipEndpoints`, além do endpoint `GET /api/dicas/adocao-parceiro`. Foram adicionadas as pastas `Resilience` e

Services, e foram usadas as pastas Endpoints, Controllers, Views/Pets, Observability e wwwroot/css. Também foram ajustados Program.cs, PetEndpoints.cs, AuthEndpoints.cs, PetsController.cs, Views/Pets/Index.cshtml, petmatch-mvc.css, PetMatchDiagnostics.cs, PetMatchMetrics.cs, appsettings.Development.json, arquivos gerados em Clients/TypeScript e documentos em docs.

O resultado esperado no navegador é a aplicação MVC abrindo em /Pets, com cards de pets e a nova faixa "cuidado responsável" exibindo uma dica de adoção. O endpoint /api/dicas/adocao-parceiro retorna a dica em JSON. O endpoint /api/pets?page=1&pageSize=2 continua retornando o envelope paginado, agora com output caching. Chamadas repetidas a endpoints protegidos por rate limiting podem retornar 429. Os Health Checks /health/live e /health/ready continuam funcionando.

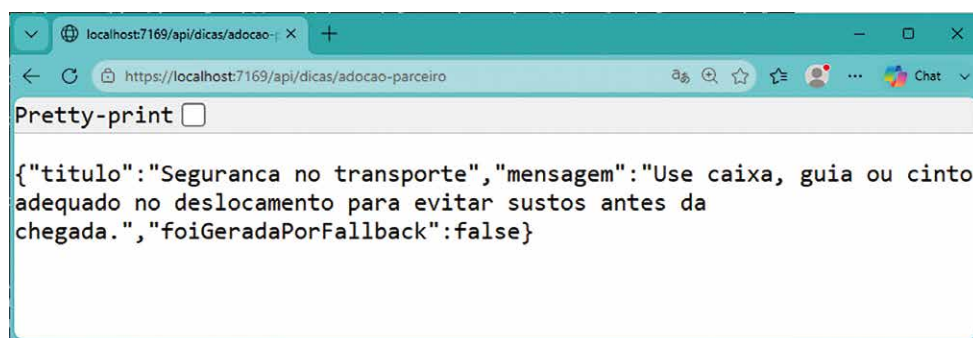


Figura 58 – Endpoint /api/dicas/adocao-parceiro com dicas de adoção em JSON

8 BUILD, IMPLANTAÇÃO E CI/CD FULL-STACK

Uma aplicação web não se torna completa apenas porque suas funcionalidades foram implementadas e executadas com sucesso durante o desenvolvimento. Entre o código escrito pelo desenvolvedor e o sistema disponível para uso real existe um percurso técnico de preparação, verificação, empacotamento, configuração e publicação. Esse percurso compõe a etapa de entrega da aplicação e envolve decisões que afetam diretamente a confiabilidade, a repetibilidade e a segurança do processo. Build, implantação e CI/CD (Continuous Integration and Continuous Delivery/Deployment) representam, nesse contexto, práticas fundamentais para transformar uma solução desenvolvida em ambiente local em um produto executável, configurado e acessível em ambientes de teste, homologação ou produção.



Saiba mais

Uma obra clássica sobre automação de build, testes e implantação, fundamental para compreender os princípios técnicos da entrega contínua, é:

HUMBLE, J.; FARLEY, D. *Continuous delivery: reliable software releases through build, test, and deployment automation*. Boston: Addison-Wesley, 2010.

Já um livro baseado em pesquisa empírica que relaciona práticas de DevOps e CI/CD ao desempenho de equipes e organizações de software é:

FORSGREN, N.; HUMBLE, J.; KIM, G. *Accelerate: the science of lean software and DevOps: building and scaling high performing technology organizations*. Portland, OR: IT Revolution Press, 2018.

O build é a etapa em que o código-fonte, as dependências, os arquivos de configuração e os recursos da aplicação são processados para gerar um resultado executável ou publicável. Em aplicações .NET, isso envolve compilar o código C#, verificar referências, preparar arquivos estáticos, aplicar configurações de projeto e produzir artefatos que possam ser executados em um ambiente de destino. Em uma aplicação full-stack, a ideia de build precisa considerar também o front-end, pois interfaces construídas com tecnologias como React passam por seu próprio processo de preparação, no qual componentes, módulos, folhas de estilo, imagens e scripts são transformados em arquivos otimizados para o navegador. O build, portanto, é a etapa que confirma se as partes da aplicação conseguem ser reunidas de modo coerente.

A implantação, por sua vez, é o processo de disponibilizar a aplicação em um ambiente no qual ela possa ser acessada e utilizada. Para publicar um sistema é necessário considerar configurações de ambiente, variáveis sensíveis, conexão com banco de dados, endereços de APIs, políticas de segurança, certificados, permissões, arquivos estáticos, logs, monitoramento e compatibilidade entre os componentes instalados. Uma aplicação pode funcionar corretamente no computador do desenvolvedor e falhar em produção se essas condições não forem tratadas com rigor. Implantar bem significa reduzir diferenças acidentais entre ambientes e tornar explícitas as dependências necessárias para que o sistema opere corretamente.

No PetMatch, essa etapa possui importância especial porque o sistema deixa de ser apenas uma construção técnica e passa a representar uma plataforma de apoio a interações reais entre pessoas, organizações e animais. Uma falha de configuração pode impedir que adotantes visualizem pets disponíveis, que uma instituição cadastre novos animais ou que manifestações de interesse sejam registradas. Uma publicação incompleta pode fazer com que a interface carregue sem consumir a API corretamente. Uma conexão mal configurada com o banco de dados pode comprometer operações essenciais. A entrega da aplicação, portanto, participa da confiabilidade do serviço e precisa ser tratada como parte integrante da arquitetura, não como um procedimento externo e improvisado.

A expressão full-stack indica que a entrega deve abranger a aplicação em sua totalidade. Em uma solução composta por API ASP.NET Core, banco de dados, interface cliente e arquivos estáticos, não basta preparar cada parte isoladamente sem compreender sua relação com as demais. A API precisa estar publicada em endereço acessível e configurada para aceitar chamadas da interface autorizada. O front-end precisa conhecer corretamente os endereços dos serviços que consome. O banco de dados precisa estar disponível, inicializado e protegido. As configurações de autenticação, autorização e comunicação entre camadas precisam permanecer coerentes. O ciclo de build e implantação deve, portanto, refletir a integração real do sistema.



Saiba mais

Uma obra prática e atual para desenvolvimento full stack com ASP.NET Core 8, Angular, Entity Framework Core, autenticação, APIs e implantação é:

DE SANCTIS, V. *ASP.NET Core 8 and Angular: full-stack web development with ASP.NET Core 8 and Angular*. 6. ed. Birmingham: Packt Publishing, 2024.

Já um bom livro voltado à integração entre ASP.NET Core, APIs REST, React, TypeScript, autenticação, testes e práticas de entrega de aplicações web completas é:

RIPPON, C. *ASP.NET Core 5 and React: full-stack web development using .NET 5, React 17, and TypeScript 4*. 2. ed. Birmingham: Packt Publishing, 2021.

O CI/CD amplia essa visão ao propor que integração e entrega sejam apoiadas por processos automatizados e repetíveis. A sigla CI refere-se à integração contínua, prática na qual alterações no código são frequentemente integradas, verificadas e validadas por meio de etapas controladas. CD pode indicar entrega contínua ou implantação contínua, dependendo do grau de automação adotado, mas em ambos os casos remete à capacidade de preparar a aplicação para publicação de modo previsível. A finalidade não é apenas acelerar a entrega, mas reduzir erros humanos, padronizar verificações, detectar problemas mais cedo e aumentar a confiança de que uma versão publicada corresponde ao código efetivamente validado.

Em projetos sem um processo disciplinado de entrega, a publicação tende a depender de ações manuais difíceis de reproduzir. Um arquivo pode ser esquecido, uma configuração pode ser alterada diretamente no servidor sem registro, uma dependência pode estar presente em um ambiente e ausente em outro, ou uma versão pode ser publicada sem que testes mínimos tenham sido executados. O CI/CD combate esse tipo de fragilidade ao transformar o caminho entre alteração de código e artefato publicável em um fluxo explícito. Esse fluxo pode verificar compilação, executar testes, preparar pacotes, organizar artefatos, aplicar configurações por ambiente e apoiar a implantação com menor margem para imprevisto.

A configuração ocupa papel central nesse processo. Uma aplicação raramente utiliza os mesmos valores nos ambientes de desenvolvimento, de homologação e de produção. Endereços de serviços, strings de conexão, chaves, políticas de log, origens permitidas, modos de diagnóstico e credenciais precisam variar conforme o ambiente. Ao mesmo tempo, essas diferenças devem ser controladas de forma segura e rastreável. Misturar configurações sensíveis ao código ou depender de ajustes manuais não documentados compromete a segurança e a previsibilidade da implantação. Em uma aplicação como o PetMatch, que pode lidar com dados de usuários, registros de interesse e informações institucionais, a separação adequada entre código e configuração é uma exigência técnica e ética.

O empacotamento também merece atenção. Para que uma aplicação seja entregue de modo consistente, é necessário definir quais artefatos representam uma versão pronta para implantação. Esses artefatos podem incluir a aplicação compilada, arquivos da interface, configurações esperadas, scripts de preparação, recursos estáticos e metadados de versão. Quando o empacotamento é claro, torna-se possível saber exatamente o que está sendo publicado. Quando é ambíguo, a implantação pode se transformar em uma combinação frágil de arquivos novos e antigos, dificultando diagnóstico e reversão. A qualidade do empacotamento influencia a capacidade de auditar, repetir e recuperar entregas.

A publicação de uma aplicação também precisa dialogar com observabilidade e resiliência. Uma nova versão deve permitir acompanhamento de seu comportamento após a implantação, pois erros podem surgir apenas sob carga real, com dados reais ou em condições específicas do ambiente. Logs, métricas e rastreamentos ajudam a confirmar se a publicação ocorreu como esperado e se a aplicação mantém desempenho e estabilidade adequados. Estratégias de reversão ou correção rápida tornam-se importantes quando uma versão apresenta falhas inesperadas. Assim, build, implantação e CI/CD não encerram o ciclo de desenvolvimento; eles se conectam à operação contínua do sistema.

No PetMatch, a entrega full-stack deve preservar a coerência da experiência do usuário e a integridade dos serviços. A pessoa que acessa a plataforma não percebe separadamente o build da API, a preparação da interface, a configuração do banco ou o fluxo de implantação. Ela percebe se o sistema está disponível, se as páginas carregam, se os dados estão corretos, se as ações são registradas e se as respostas são confiáveis. Por isso, a entrega técnica precisa ser pensada a partir da aplicação como um todo. Cada camada possui seus próprios requisitos, mas o valor final depende da integração entre elas.

Este tópico introduz build, implantação e o CI/CD como partes essenciais do ciclo técnico de entrega em aplicações web com .NET. Depois de construir funcionalidades, persistir dados, expor APIs, integrar interfaces, proteger recursos e observar a aplicação em execução, torna-se necessário compreender como tudo isso é preparado e publicado de modo confiável. A entrega não é um evento isolado no fim do projeto, mas um processo contínuo que conecta desenvolvimento, configuração, empacotamento, validação e operação. Ao tratar o PetMatch como uma aplicação completa, o leitor passa a perceber que desenvolver software para a web garantir que os recursos funcionais criados pelo desenvolvedor possam chegar ao usuário final com estabilidade, segurança e repetibilidade.

8.1 Configuração por ambientes, secrets e publicação containerizada

Uma aplicação web profissional não deve depender de valores fixos no código-fonte para funcionar corretamente. Endereços de serviços externos, strings de conexão, níveis de log, opções de cache, políticas operacionais e chaves de integração variam conforme o ambiente em que a aplicação está sendo executada. O ambiente de desenvolvimento costuma privilegiar diagnóstico, facilidade de execução local e dados controlados; o ambiente de homologação procura reproduzir condições próximas da produção; o ambiente de produção exige configuração segura, menor exposição de detalhes internos e cuidado com disponibilidade. Separar configuração do código permite que a mesma aplicação seja executada em contextos diferentes sem recompilar a solução e sem alterar classes apenas para trocar valores operacionais.

No ASP.NET Core, a configuração é composta por várias fontes. Arquivos como `appsettings.json` e `appsettings.Development.json` fornecem valores estruturados em JSON. Arquivos específicos de ambiente, como `appsettings.Staging.json` e `appsettings.Production.json`, permitem especializar o comportamento da aplicação conforme o valor de ambiente carregado. Variáveis de ambiente podem sobrescrever valores de arquivos e são importantes em servidores e containers. `User Secrets` permitem armazenar valores sensíveis no ambiente local de desenvolvimento, fora do repositório. O ponto central é que a aplicação consulta uma configuração unificada, mas essa configuração é montada a partir de fontes diferentes, com precedência e finalidade próprias.

Um secret é uma configuração sensível. Senhas, tokens, chaves de API e strings de conexão reais pertencem a essa categoria porque sua exposição pode comprometer dados, acesso ou infraestrutura. Nem toda configuração é secret: o nome público da aplicação, um tempo de expiração de cache ou uma mensagem visual segura podem estar em `appsettings.json`. Um secret, por outro lado, não deve ser gravado em arquivos versionados, não deve aparecer em prints de tela, não deve ser colocado no `Dockerfile` e não deve ser copiado para a imagem do container. Em desenvolvimento, o Visual Studio 2026 oferece a opção `Manage User Secrets`, que permite manter esses valores no perfil local do desenvolvedor, associados ao projeto, mas fora do código-fonte.

As variáveis de ambiente são uma forma externa de fornecer valores à aplicação. Elas são fundamentais em ambientes de hospedagem e em containers porque permitem executar a mesma imagem com configurações diferentes. Em ASP.NET Core, a hierarquia de configuração pode ser representada com dois underlines. Assim, uma opção como `PetMatch:AdoptionTips:ApiKey` pode ser fornecida como `PetMatch__AdoptionTips__ApiKey`. Essa convenção é importante porque containers são projetados para serem reproduzíveis: a imagem descreve a aplicação e seu runtime, enquanto o ambiente fornece os valores que mudam entre desenvolvimento, homologação e produção.

Os containers resolvem outro aspecto da preparação operacional: a reprodutibilidade do ambiente de execução. Um `Dockerfile` descreve como a aplicação é construída, publicada e executada dentro de uma imagem. Isso reduz a dependência de configurações manuais da máquina do desenvolvedor e aproxima a aplicação de um processo de entrega previsível. O container, entretanto, não elimina a necessidade de configuração segura. Ao contrário, ele torna essa disciplina mais importante, pois uma imagem pode ser compartilhada e executada em vários lugares. Por isso, o `Dockerfile` deve conter instruções de build e runtime, mas não deve conter secrets.

No PetMatch, esta etapa mantém a aplicação em sua fase unificada, centrada no projeto `PetMatch.Web`. A área MVC continua sendo a interface principal do usuário, com a rota `/Pets`, cards de pets, navegação, login, cadastro, páginas protegidas e a faixa de cuidado responsável. Minimal APIs, `Health Checks`, logging, métricas, tracing, rate limiting e output caching permanecem no mesmo projeto. A evolução deste subtópico não cria uma tela técnica de configuração; ela prepara a aplicação para ser configurada com segurança e executada de forma mais previsível em diferentes ambientes.

A estrutura criada para organizar a configuração começa pela pasta `Configuration`, no projeto `PetMatch.Web`. O arquivo `PetMatch.Web/Configuration/PetMatchApplicationOptions.cs`

representa valores públicos e seguros da aplicação. O objetivo dessa classe é demonstrar o uso de opções tipadas, isto é, classes C# que recebem valores vindos da configuração sem que controllers e serviços precisem ler arquivos diretamente.

```
namespace PetMatch.Web.Configuration;

public sealed class PetMatchApplicationOptions
{
    public const string SectionName = "PetMatch:Application";

    public string Name { get; init; } = "PetMatch";
    public string PublicMessage { get; init; } = "Adoção responsável começa com informação clara.";
}
```

Esse código define uma seção de configuração e valores padrão seguros. Name e PublicMessage são exemplos de opções que podem ser expostas sem comprometer a aplicação. Elas ajudam a diferenciar o que é configuração operacional pública daquilo que deve ser tratado como segredo.

A configuração da integração de dicas de adoção responsável fica em PetMatch.Web/Configuration/AdoptionTipsOptions.cs. Essa classe concentra opções do serviço de dicas, incluindo URL, caminho, timeout, tentativas, limite do circuit breaker e nome de cabeçalho. A propriedade ApiKey é opcional e deve ser fornecida por User Secrets em desenvolvimento ou por variável de ambiente em execução containerizada.

```
namespace PetMatch.Web.Configuration;

public sealed class AdoptionTipsOptions
{
    public const string SectionName = "PetMatch:AdoptionTips";

    public string BaseUrl { get; init; } = "https://localhost";
    public string Path { get; init; } = "/api/dicas/adocao-parceiro";
    public int TimeoutSeconds { get; init; } = 3;
    public int RetryAttempts { get; init; } = 3;
    public int CircuitBreakerFailures { get; init; } = 3;
    public string ApiKeyHeaderName { get; init; } = "X-PetMatch-Tips-Key";
    public string? ApiKey { get; init; }
}
```

Essa classe mostra uma separação essencial: a aplicação conhece o formato da configuração, mas o valor secreto não precisa estar em arquivo versionado. O nome do cabeçalho pode ser público; a chave real não. O leitor deve copiar a estrutura, mas nunca gravar uma chave real em appsettings.json, appsettings.Staging.json, appsettings.Production.json, Dockerfile ou perfil versionado.

O arquivo PetMatch.Web/appsettings.json passa a conter apenas valores comuns e seguros. Ele é a base da configuração e pode definir opções que não dependem de credenciais.

```
{
  "PetMatch": {
    "Application": {
      "Name": "PetMatch",
      "PublicMessage": "Adoção responsável começa com informação clara."
    },
    "AdoptionTips": {
      "BaseUrl": "https://localhost",
      "Path": "/api/dicas/adocao-parceiro",
      "TimeoutSeconds": 3,
      "RetryAttempts": 3,
      "CircuitBreakerFailures": 3,
      "ApiKeyHeaderName": "X-PetMatch-Tips-Key"
    }
  }
}
```

Esse trecho demonstra a configuração comum da aplicação. Não há senha, token ou chave de API. A seção `PetMatch:Application` contém dados públicos e a seção `PetMatch:AdoptionTips` contém parâmetros operacionais seguros. A propriedade `ApiKey` não aparece nesse arquivo. O arquivo `PetMatch.Web/appsettings.Development.json` mantém valores adequados à execução local, incluindo logs e banco SQLite de desenvolvimento. Ele pode especializar a configuração sem comprometer a segurança.

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning",
      "PetMatch.Web": "Information"
    }
  },
  "ConnectionStrings": {
    "PetMatchConnection": "Data Source=petmatch-dev.db"
  }
}
```

Esse arquivo é voltado ao desenvolvimento. A string SQLite local é apropriada para o exemplo didático porque não contém segredo real de infraestrutura. Em um banco remoto com usuário e senha, a string de conexão sensível deveria ser tratada como `secret`.

Os arquivos `PetMatch.Web/appsettings.Staging.json` e `PetMatch.Web/appsettings.Production.json` demonstram a separação de ambientes sem incluir credenciais reais. Eles devem conter placeholders seguros ou valores públicos, deixando secrets para variáveis de ambiente ou mecanismo seguro do provedor.

```
{
  "PetMatch": {
    "Application": {
      "Name": "PetMatch Homologação",

```

```

    "PublicMessage": "Ambiente de homologação para validação operacional."
  },
  "AdoptionTips": {
    "BaseUrl": "https://homologacao.exemplo-seguro.invalid",
    "Path": "/api/dicas/adocao-parceiro"
  }
}
}

```

Esse exemplo representa homologação com um endereço ilustrativo e seguro. O valor não é uma credencial. A função do arquivo é demonstrar a variação por ambiente, não fornecer dados reais de operação.

```

{
  "PetMatch": {
    "Application": {
      "Name": "PetMatch",
      "PublicMessage": "Cuidado responsável para adoção de pets."
    },
    "AdoptionTips": {
      "BaseUrl": "https://producao.exemplo-seguro.invalid",
      "Path": "/api/dicas/adocao-parceiro"
    }
  }
}

```

Esse exemplo representa produção sem expor chave de API, senha ou token. Em um ambiente real, a chave de integração seria fornecida externamente por variável de ambiente, como `PetMatch__AdoptionTips__ApiKey` ou por um serviço seguro de configuração.

O registro das opções ocorre em `PetMatch.Web/Program.cs`. Esse arquivo preserva a configuração já existente da aplicação, como MVC, Identity, Minimal APIs, Health Checks, observabilidade, rate limiting e output caching, e acrescenta o binding das opções tipadas.

```

builder.Services.Configure<PetMatchApplicationOptions>(
    builder.Configuration.GetSection(PetMatchApplicationOptions.SectionName));

builder.Services.Configure<AdoptionTipsOptions>(
    builder.Configuration.GetSection(AdoptionTipsOptions.SectionName));

```

Esse código conecta as seções de configuração às classes C#. A partir desse ponto, controllers e serviços podem receber `IOptions<T>` ou `IOptionsMonitor<T>` pela injeção de dependência. Isso reduz acoplamento, evita leitura manual de arquivos e torna o código mais testável e organizado.

O serviço `PetMatch.Web/Services/AdoptionTipClient.cs`, criado no subtópico anterior para demonstrar resiliência, passa a consumir `AdoptionTipsOptions`. Assim, a URL, o caminho, o timeout e a chave opcional deixam de ser valores fixos no código.


```
public sealed class AdoptionTipClient(
    HttpClient httpClient,
    IOptionMonitor<AdoptionTipsOptions> options,
    ILogger<AdoptionTipClient> logger)
{
    private readonly HttpClient _httpClient = httpClient;
    private readonly IOptionMonitor<AdoptionTipsOptions> _options = options;
    private readonly ILogger<AdoptionTipClient> _logger = logger;

    public async Task<AdoptionTipResponse> GetTipAsync(CancellationToken
cancellationToken)
    {
        var currentOptions = _options.CurrentValue;

        if (!string.IsNullOrEmpty(currentOptions.ApiKey))
        {
            _httpClient.DefaultRequestHeaders.Remove(currentOptions.
ApiKeyHeaderName);

            _httpClient.DefaultRequestHeaders.Add(
                currentOptions.ApiKeyHeaderName,
                currentOptions.ApiKey);
        }

        // restante da chamada resiliente preservado
        return await BuscarDicaComFallbackAsync(currentOptions,
cancellationToken);
    }
}
```

Esse trecho mostra o princípio implementado: o serviço recebe opções por injeção de dependência e usa a chave somente se ela estiver disponível. A chave pode vir de `User Secrets` no desenvolvimento ou de variável de ambiente no container. O código não exige que o secret exista em arquivo versionado.

A tela `/Pets` continua sendo uma página de usuário. O arquivo `PetMatch.Web/Views/Pets/Index.cshtml` foi ajustado para exibir uma mensagem pública configurável e a faixa de dica de adoção responsável, sem expor ambiente técnico, secrets ou dados internos.

```
@if (!string.IsNullOrEmpty(ViewBag.PublicMessage))
{
    <section class="public-message">
        <p>@ViewBag.PublicMessage</p>
    </section>
}
```

Esse trecho exibe apenas uma mensagem pública segura. O leitor deve observar que a configuração pode influenciar o conteúdo visual da aplicação sem revelar detalhes operacionais.

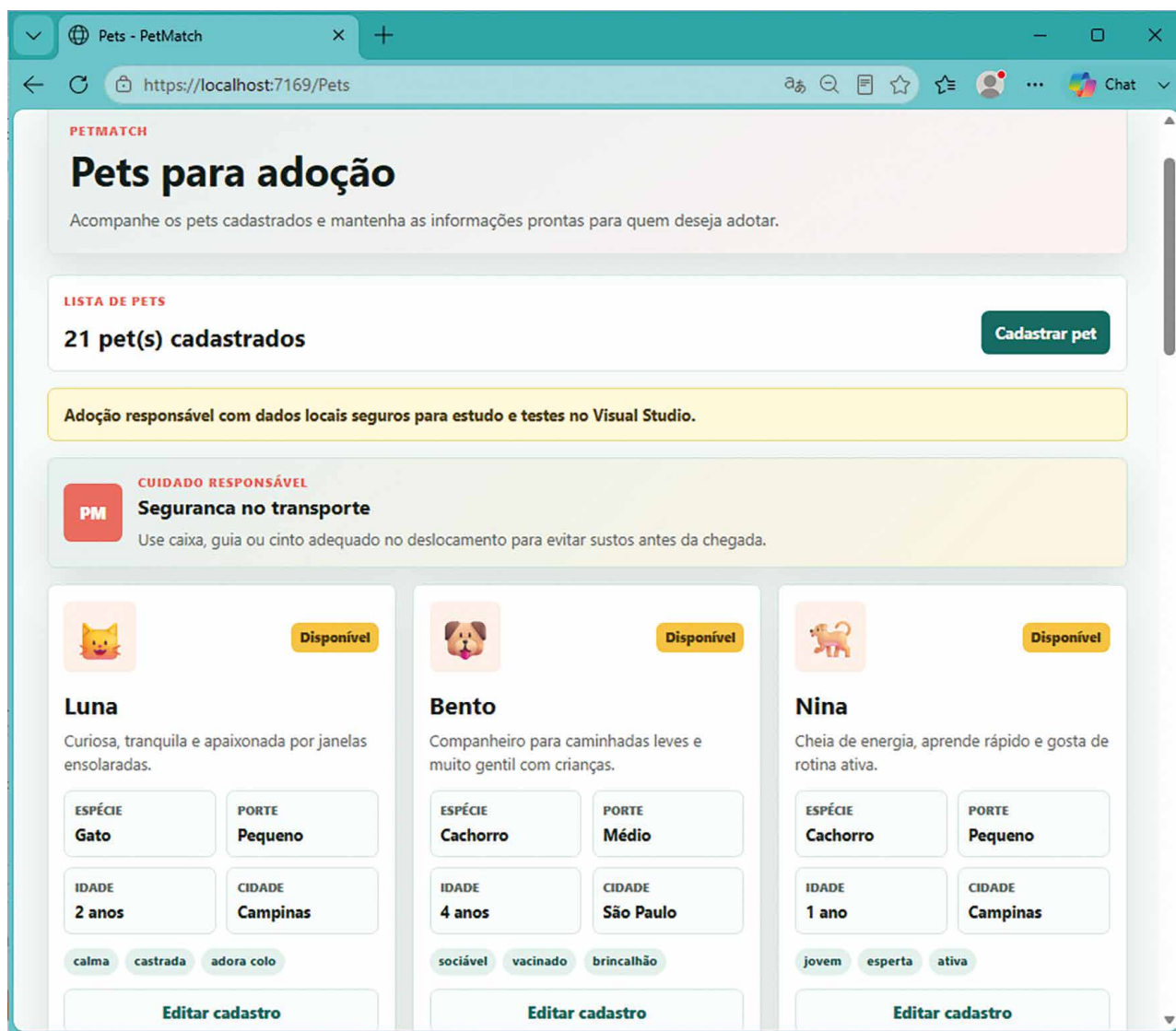


Figura 59 – Página /Pets aberta no navegador, mostrando os cards de pets, a navegação e a mensagem pública configurada junto à faixa de cuidado responsável

O projeto também recebeu `UserSecretsId` em `PetMatch.Web.csproj`, habilitando o fluxo `Manage User Secrets` pelo Visual Studio 2026. O objetivo é permitir que o desenvolvedor armazene localmente um valor fictício para `PetMatch:AdoptionTips:ApiKey`, sem colocar esse valor no repositório.

```
<PropertyGroup>
  <UserSecretsId>petmatch-web-development-secrets</UserSecretsId>
</PropertyGroup>
```

Esse identificador associa o projeto a um arquivo local de secrets mantido fora da pasta do projeto. O valor usado no livro é fictício. A prática correta é entender o mecanismo e não copiar credenciais reais. A containerização foi preparada com um Dockerfile em `PetMatch.Web/Dockerfile`. Esse arquivo descreve como restaurar, publicar e executar a aplicação ASP.NET Core dentro de uma imagem compatível com .NET 10.

```
FROM mcr.microsoft.com/dotnet/aspnet:10.0 AS base
WORKDIR /app
EXPOSE 8080
FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
WORKDIR /src
COPY ["PetMatch.Web.csproj", "."]
RUN dotnet restore "PetMatch.Web.csproj"
COPY..
RUN dotnet publish "PetMatch.Web.csproj" -c Release -o /app/publish

FROM base AS final
WORKDIR /app
COPY --from=build /app/publish.
ENTRYPOINT ["dotnet", "PetMatch.Web.dll"]
```

Esse Dockerfile usa uma imagem de SDK para construir a aplicação e uma imagem de runtime para executá-la. O arquivo não contém senha, token, chave de API ou string de conexão real. A configuração sensível deve entrar no container por variáveis de ambiente ou mecanismo seguro do ambiente de execução. O arquivo `PetMatch.Web/.dockerignore` reduz o contexto enviado para a construção da imagem. Ele impede que pastas de `build`, arquivos temporários e conteúdos desnecessários sejam copiados para o processo de containerização.

```
bin/
obj/
.vs/
*.user
*.suo
node_modules/
```

Esse arquivo melhora a organização do build e reduz o risco de incluir conteúdo local indevido. Ele complementa o Dockerfile: enquanto o Dockerfile descreve como construir a imagem, o `.dockerignore` define o que não deve entrar no contexto de build. O perfil `Container` (Dockerfile) foi adicionado em `PetMatch.Web/Properties/launchSettings.json`, permitindo que o Visual Studio 2026 ofereça um fluxo visual para execução containerizada quando o ambiente possuir suporte a containers. A publicação por assistente não criou `PetMatchContainer.pubxml`; o projeto ficou preparado com Dockerfile, `.dockerignore` e perfil de container.

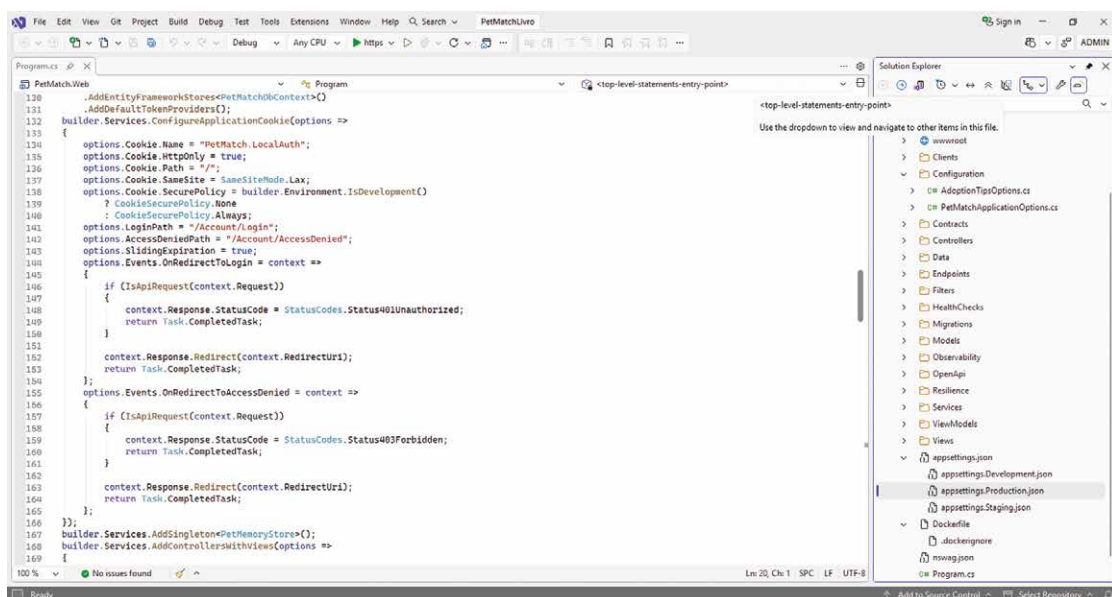


Figura 60 – Gerenciador de soluções do Visual Studio 2026 mostrando a pasta Configuration, os arquivos appsettings.Staging.json, appsettings.Production.json, o Dockerfile e o .dockerignore

O resultado técnico esperado é a aplicação continuar funcionando em execução local, com /Pets, /health/live, /health/ready e /api/dicas/adocao-parceiro. O resultado operacional é mais importante: a aplicação passa a ter configuração por ambiente, opções tipadas, User Secrets habilitado, Dockerfile preparado e ausência de secrets reais em arquivos versionados.

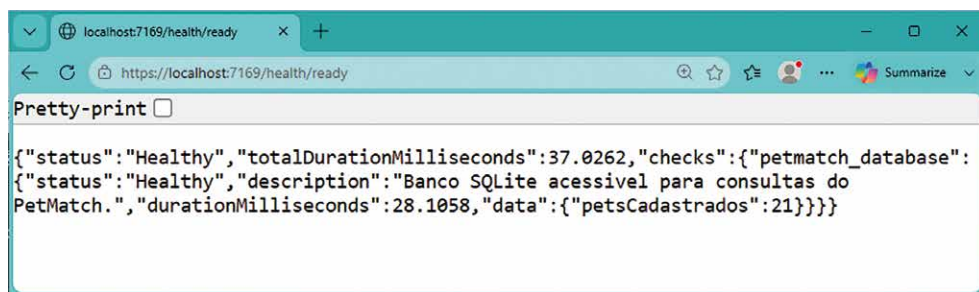


Figura 61 – Página /health/ready aberta no navegador, mostrando resposta saudável após a reorganização de configuração e a preparação para container

A implementação criou:

- Configuration/PetMatchApplicationOptions.cs
- Configuration/AdoptionTipsOptions.cs
- appsettings.Staging.json
- appsettings.Production.json

- Dockerfile
- .dockerignore

Também foram modificados:

- Program.cs
- AdoptionTipClient.cs
- PetsController.cs
- Views/Pets/Index.cshtml
- PetMatch.Web.csproj
- Properties/launchSettings.json
- appsettings.json
- appsettings.Development.json
- documentos técnicos em docs

8.2 Deploy integrado (API servindo estáticos do React) ou pipelines separados (API + front-end)

A entrega de uma aplicação full-stack não se limita a copiar arquivos para um servidor. Quando um sistema possui back-end, API, páginas MVC, autenticação, configuração por ambientes, Health Checks, políticas de resiliência e uma experiência front-end construída em React, a publicação passa a envolver decisões de arquitetura operacional. É necessário definir se o back-end e o front-end serão entregues juntos, em um único artefato, ou separadamente, com pipelines, ambientes e ciclos de publicação distintos. Essa decisão afeta versionamento, domínio, cache, rollback, segurança, variáveis de ambiente, roteamento e responsabilidades de manutenção.

No deploy integrado, a aplicação ASP.NET Core publica a API e também serve os arquivos estáticos gerados pelo front-end React. O React, nesse modelo, deixa de ser executado por um servidor de desenvolvimento como Vite e passa a existir como um conjunto de arquivos estáticos: HTML, CSS, JavaScript e assets. Esses arquivos são colocados em uma pasta pública do projeto ASP.NET Core, e o próprio `PetMatch.Web` passa a entregá-los ao navegador. A vantagem principal é a simplicidade operacional: uma aplicação, um host, um domínio, uma configuração central e menor exposição a problemas de CORS. A desvantagem é o acoplamento entre os ciclos de entrega: uma alteração no front-end pode exigir a publicação do pacote completo e uma falha no pipeline integrado pode bloquear a entrega do conjunto.

No deploy separado, API e front-end seguem caminhos independentes. A API ASP.NET Core pode ser publicada como aplicação web ou container, enquanto o React pode ser hospedado como site estático em um serviço especializado ou CDN. Essa abordagem favorece equipes separadas, cache agressivo de arquivos estáticos, escalabilidade independente e rollback isolado. Em contrapartida, exige cuidado maior com URLs da API, CORS, cookies, autenticação, variáveis de ambiente do front-end, compatibilidade entre versões e coordenação dos pipelines. Assim, a decisão não é uma escolha puramente tecnológica; é uma decisão de entrega, operação e governança.

No PetMatch, a fase operacional permanece centrada em `PetMatch.Web`. O projeto React construído anteriormente continua preservado como origem possível de artefatos estáticos, mas não volta a ser dependência da execução local diária. A aplicação principal continua abrindo em `/Pets`, com a interface MVC, cards de pets, aviso configurável, dica de adoção responsável, autenticação, `Health Checks` e API pública. O objetivo desta etapa é preparar os dois caminhos de entrega sem transformar a prática local em um fluxo dependente de front-end separado.

O `PetMatch.Web` continua sendo o projeto principal da solução ativa. A rota `/app` fica reservada para uma possível vitrine React integrada, servida pelo próprio ASP.NET Core quando houver um arquivo real em `wwwroot/app/index.html`. Enquanto esse artefato não existir, a aplicação não simula uma tela inexistente: `/app` retorna 404 e a experiência local permanece em `/Pets`. Esse comportamento evita confusão para o leitor, pois separa claramente a preparação para deploy integrado da execução visual atual.

Para representar essa decisão no código, foi criada a classe `FrontendDeploymentOptions`, no projeto `PetMatch.Web`, em `Configuration/FrontendDeploymentOptions.cs`. Ela concentra opções seguras sobre o modo de entrega do front-end, o caminho integrado, a pasta de arquivos estáticos, o caminho base da API e o arquivo inicial da aplicação React.

```
namespace PetMatch.Web.Configuration;

public sealed class FrontendDeploymentOptions
{
    public const string SectionName = "PetMatch:FrontendDeployment";

    public string Mode { get; init; } = "Integrated";
    public string IntegratedPath { get; init; } = "/app";
    public string StaticFilesPath { get; init; } = "wwwroot/app";
    public string ApiBasePath { get; init; } = "/api";
    public string IndexFile { get; init; } = "index.html";
}
```

Esse código não executa a publicação por si só. Seu papel é tornar explícita a escolha operacional dentro da configuração da aplicação. Com isso, o modo de entrega deixa de ser um detalhe informal e passa a ser uma opção nomeada, legível e segura. Não há secrets nessa classe, pois o modo de publicação, o caminho da SPA e o caminho da API são informações operacionais, não credenciais.

Os arquivos `appsettings.json`, `appsettings.Development.json`, `appsettings.Staging.json` e `appsettings.Production.json` foram ajustados para incluir opções seguras de entrega. O trecho a seguir ilustra a configuração comum do modo integrado.

```
{
  "PetMatch": {
    "FrontendDeployment": {
      "Mode": "Integrated",
      "IntegratedPath": "/app",
      "StaticFilesPath": "wwwroot/app",
      "ApiBasePath": "/api",
      "IndexFile": "index.html"
    }
  }
}
```

Essa configuração informa que, quando os arquivos do React forem integrados, eles serão servidos a partir de `/app`. O caminho `/api` permanece reservado às Minimal APIs e `/pets` continua sendo a área MVC principal. A separação dos caminhos é importante porque o fallback da SPA não deve capturar rotas do servidor, endpoints de API ou Health Checks.

A pasta `PetMatch.Web/wwwroot/app` foi criada para receber os artefatos estáticos do React quando a estratégia integrada for usada. O arquivo `.gitkeep` mantém a pasta no repositório mesmo quando ela ainda não possui `index.html`, CSS ou JavaScript. Essa pasta não contém o código-fonte React, apenas o destino de publicação dos arquivos já gerados pelo build do front-end.

O mapeamento no `Program.cs`, no projeto `PetMatch.Web`, foi ajustado para servir a SPA integrada apenas quando o arquivo inicial existir. O trecho essencial preserva a aplicação MVC e a API, restringindo o fallback da SPA ao caminho `/app`.

```
var frontendOptions = app.Services
    .GetRequiredService<IOptions<FrontendDeploymentOptions>>()
    .Value;

var integratedIndexPath = Path.Combine(
    app.Environment.WebRootPath,
    "app",
    frontendOptions.IndexFile);

if (File.Exists(integratedIndexPath))
{
    app.MapFallbackToFile(
        $"{frontendOptions.IntegratedPath}/{*path:nonfile}",
        $"app/{frontendOptions.IndexFile}");
}
```

Esse código produz um comportamento seguro para a fase atual do `PetMatch`. Se `wwwroot/app/index.html` existir, o ASP.NET Core serve a aplicação React integrada em `/app`. Se o arquivo não existir, a rota não é criada e `/app` retorna 404. Com isso, `/pets`, `/api/pets`, `/health/live` e `/health/ready` continuam funcionando sem serem capturados pelo fallback da SPA.

O arquivo `PetMatch.Web.csproj` recebeu o alvo opcional `CopyIntegratedFrontendAssets`, desligado por padrão. A finalidade é documentar o ponto de integração entre o build do React e a publicação do ASP.NET Core sem obrigar o leitor a executar o front-end na prática local. O alvo só deve ser usado quando houver artefatos reais do React prontos para cópia.

```
<Target Name="CopyIntegratedFrontendAssets"
  BeforeTargets="ComputeFilesToPublish"
  Condition="'$(UseIntegratedFrontendAssets)' == 'true'">
  <ItemGroup>
    <IntegratedFrontendFiles Include="..\PetMatch.Frontend\dist\**\*.*" />
  </ItemGroup>

  <Copy SourceFiles="@ (IntegratedFrontendFiles)"
    DestinationFolder="wwwroot\app\%(RecursiveDir)" />
</Target>
```

Esse trecho representa o ponto de acoplamento no deploy integrado. Ele não reconstrói o React, não executa servidor de desenvolvimento e não cria dependência obrigatória no fluxo local. Apenas permite copiar arquivos estáticos já gerados para `wwwroot/app` quando a propriedade `UseIntegratedFrontendAssets` estiver habilitada no processo de publicação.

O `Dockerfile` criado anteriormente foi mantido compatível com a publicação do `PetMatch.Web`, preservando a pasta `wwwroot` no resultado final. Como `wwwroot/app` pertence ao projeto web, os artefatos estáticos integrados, quando presentes, seguem junto com o publish da aplicação. O `Dockerfile` não contém secrets, não armazena chaves e não resolve configuração sensível; ele apenas empacota a aplicação publicada.

Além da preparação integrada, o repositório recebeu documentação para os dois modelos de entrega. O arquivo `deployment/integrated/README.md` descreve a estratégia em que o `PetMatch.Web` publica API, MVC e arquivos estáticos integrados. O arquivo `deployment/separated/README.md` descreve a alternativa em que API e front-end seguem pipelines próprios. Esses documentos não são telas da aplicação; eles registram decisões técnicas do projeto para orientar manutenção e entrega.

Também foram criados três workflows em `.github/workflows`: `petmatch-integrated.yml`, `petmatch-api.yml` e `petmatch-frontend.yml`. O primeiro representa a entrega integrada do `PetMatch.Web`. O segundo representa um pipeline separado para a API. O terceiro representa um pipeline separado para o front-end. Esses arquivos são modelos de CI/CD e não precisam ser executados localmente pelo leitor. O ponto mais importante é que eles não contêm secrets reais; qualquer credencial necessária deve aparecer apenas como referência simbólica a secrets do ambiente de CI.

```
name: PetMatch Integrated

on:
  workflow_dispatch:

jobs:
  build:
    runs-on: ubuntu-latest
```

```
steps:
  - name: Checkout
    uses: actions/checkout@v4
```

Esse trecho ilustra a estrutura de um pipeline como configuração versionada. Ele descreve etapas de entrega, mas não carrega senhas, tokens ou chaves de API. O mesmo princípio vale para os pipelines separados: os arquivos podem referenciar nomes de secrets do ambiente, mas nunca devem armazenar valores sensíveis diretamente.

Ao executar `PetMatch.Web` pelo Visual Studio 2026, o navegador abre a experiência MVC em `/Pets`. A API continua respondendo em `/api/pets?page=1&pageSize=2`, e os Health Checks continuam saudáveis em `/health/live` e `/health/ready`. Como ainda não há `wwwroot/app/index.html`, `/app` retorna 404, comportamento esperado e intencional. Quando os arquivos estáticos reais do React forem copiados para `wwwroot/app`, essa mesma rota passará a servir a vitrine integrada pelo ASP.NET Core.

A implementação criou:

- `Configuration/FrontendDeploymentOptions.cs`
- `wwwroot/app/.gitkeep`
- `deployment/integrated/README.md`
- `deployment/separated/README.md`
- `.github/workflows/petmatch-integrated.yml`
- `.github/workflows/petmatch-api.yml`
- `.github/workflows/petmatch-frontend.yml`

Também foram modificados:

- `Program.cs`
- `PetMatch.Web.csproj`
- `Dockerfile`,
- arquivos `appsettings*.json`
- documentos técnicos em docs.

O `PetMatch.Frontend` não voltou para a solução ativa.



Resumo

O tópico 7 desloca o foco da implementação funcional para a qualidade operacional da aplicação, apresentando observabilidade e resiliência como propriedades essenciais em sistemas web sujeitos a falhas de rede, variações de carga, lentidão, indisponibilidade de dependências e exceções difíceis de reproduzir.

A observabilidade é definida como a capacidade de compreender o comportamento do sistema por meio de sinais úteis e contextualizados, como logs, métricas, tracing e Health Checks, permitindo investigar eventos, acompanhar tendências, verificar vida e prontidão da aplicação e diagnosticar problemas sem expor dados sensíveis.

A resiliência, por sua vez, refere-se à capacidade de manter respostas adequadas diante de falhas ou atrasos, recorrendo a estratégias como tratamento de exceções, timeouts, retry controlado, circuit breaker, rate limiting, degradação graciosa e output caching. Assim, o tópico reforça que desenvolver bem não significa apenas entregar funcionalidades, mas construir sistemas observáveis, diagnosticáveis e capazes de sustentar seu funcionamento com previsibilidade, controle e recuperação.

O tópico 8 trata da entrega de aplicações web full-stack como etapa arquitetural que envolve preparação, verificação, empacotamento, configuração e publicação em ambientes reais. O build é apresentado como o processo que reúne código, dependências e recursos em artefatos executáveis ou publicáveis, incluindo tanto a aplicação .NET quanto o front-end quando houver interfaces como React.

A implantação é compreendida como a disponibilização controlada da aplicação em desenvolvimento, homologação ou produção, exigindo atenção a banco de dados, certificados, permissões, arquivos estáticos, logs, monitoramento e segurança.

O tópico também aborda CI/CD como fluxo automatizado e repetível para reduzir erros manuais, padronizar validações e aumentar a confiança entre código versionado e artefato publicado. Por fim, discute configuração por ambientes, User Secrets, variáveis de ambiente, opções tipadas, containerização e estratégias de deploy integrado ou separado, consolidando build, implantação e CI/CD como práticas que conectam desenvolvimento, segurança, empacotamento, observabilidade e operação contínua.



Exercícios

Questão 1. Uma empresa de transporte urbano mantém uma aplicação ASP.NET Core para consulta de linhas, compra de passagens e gestão de contas. Após períodos de instabilidade, a equipe identificou três situações recorrentes: lentidão na consulta de horários, falhas temporárias em um serviço externo de localização e grande volume de tentativas de login.

A arquitetura apresenta os seguintes recursos:

- o endpoint GET /health/live verifica se o processo está ativo;
- o endpoint GET /health/ready verifica o banco de dados e as dependências essenciais;
- as consultas públicas de horários podem ser armazenadas temporariamente;
- o serviço externo de localização é usado apenas em operações de leitura;
- as páginas de perfil e o histórico de viagens dependem da identidade autenticada;
- os eventos são registrados por meio de logs estruturados;
- contadores medem consultas, falhas e tentativas de login;
- atividades de tracing acompanham o percurso entre endpoint, banco de dados e serviço externo.

Durante a revisão da arquitetura, foram apresentadas diferentes propostas para ampliar a observabilidade e a resiliência do sistema.

Qual plano técnico deve ser aprovado?

- A) A verificação de vida deve consultar o banco e o serviço de localização. A verificação de prontidão deve examinar apenas o processo. As leituras externas podem receber retry limitado, as respostas públicas podem ser armazenadas em cache e os sinais de observabilidade devem permanecer correlacionados.
- B) A verificação de vida deve examinar a capacidade básica do processo. A prontidão deve considerar as dependências essenciais. O retry deve ser limitado a leituras sujeitas a falhas transitórias e o circuit breaker deve conter chamadas repetidamente malsucedidas. O plano deve incluir cache público, limites específicos para login e telemetria sem dados sensíveis.

- C) As verificações de vida e prontidão devem permanecer separadas. O serviço de localização pode usar retry e circuit breaker. A mesma política de retry deve ser aplicada às compras de passagens, pois uma nova tentativa após timeout é necessária para descobrir se a transação anterior foi concluída.
- D) A arquitetura deve combinar logs, métricas e tracing, além de separar vida de prontidão e limitar as tentativas de login. Para facilitar o diagnóstico, os logs estruturados devem registrar integralmente os corpos das requisições, incluindo credenciais, tokens e respostas do serviço externo.
- E) A solução deve manter vida e prontidão separadas, aplicar retry somente às leituras e restringir o cache a dados públicos. Uma única política de rate limiting deve atender a todos os endpoints e qualquer excesso deve produzir 503 Service Unavailable para indicar indisponibilidade temporária.

Resposta correta: alternativa B.

Análise das alternativas

A) Alternativa incorreta.

Justificativa: a verificação de vida responde se a instância ainda consegue executar e deve permanecer ativa. Se ela depender do banco ou de um serviço externo, uma falha dessas dependências pode provocar reinicializações de processos saudáveis. A prontidão é o local adequado para indicar que a instância não deve receber tráfego enquanto uma dependência essencial estiver indisponível.

B) Alternativa correta.

Justificativa: a separação entre vida e prontidão evita confundir falha do processo com indisponibilidade de uma dependência. A repetição de uma leitura tende a não produzir efeitos duplicados, enquanto o circuito aberto reduz a pressão sobre um serviço que continua falhando. Os limites de requisição podem ser definidos segundo o custo e a sensibilidade de cada operação. Logs, métricas e traces respondem a perguntas diferentes sobre eventos, tendências e percursos.

C) Alternativa incorreta.

Justificativa: depois de um timeout, o cliente pode desconhecer se a compra foi processada. Repetir a operação sem uma chave de idempotência ou outro mecanismo de deduplicação pode criar duas passagens ou duas cobranças. A segurança do retry depende das propriedades da operação, e não apenas do tipo de falha observado.

D) Alternativa incorreta.

Justificativa: a correlação dos sinais pode ser realizada com identificadores de requisição, tempos, categorias e resultados. Senhas e tokens não são necessários para relacionar os eventos e ampliam o

dano causado por acesso indevido aos registros. Dados sensíveis devem ser excluídos ou mascarados antes da gravação.

E) Alternativa incorreta.

Justificativa: endpoints de consulta, autenticação e administração possuem custos e riscos distintos, o que pode exigir limites diferentes. O status 429 Too Many Requests informa que o cliente excedeu a quantidade permitida. O status 503 indica que o serviço está temporariamente incapaz de atender, situação diferente da limitação aplicada a um solicitante.

Questão 2. Uma instituição de ensino desenvolveu uma plataforma para matrícula em cursos. A solução reúne uma aplicação ASP.NET Core, uma API, páginas MVC, autenticação, um banco de dados e uma interface React. A equipe precisa organizar a publicação nos ambientes de desenvolvimento, homologação e produção.

A arquitetura apresenta as seguintes condições:

- os endereços de serviços, os níveis de log e as mensagens públicas variam conforme o ambiente;
- a string de conexão de produção e a chave de uma integração externa são informações sensíveis;
- a aplicação será publicada em uma imagem de container;
- o React pode ser entregue pelo próprio ASP.NET Core, em /app, ou hospedado separadamente;
- os endpoints /api, /health/live e /health/ready devem permanecer acessíveis;
- a interface React integrada somente deve ser servida quando o arquivo wwwroot/app/index.html estiver presente;
- os pipelines de integração e entrega contínuas serão mantidos em arquivos versionados;
- a mesma imagem de container deverá ser utilizável em homologação e produção.

Durante o planejamento, a equipe analisou como tratar as configurações, os secrets, o build e as duas estratégias de implantação.

Assinale a alternativa que apresenta um plano tecnicamente adequado para essa entrega.

- A) As configurações públicas podem permanecer em appsettings e os valores sensíveis podem ser fornecidos por variáveis de ambiente. O container deve usar múltiplas etapas, mas o pipeline precisa gerar uma imagem própria para homologação e outra para produção, incorporando durante o build as configurações públicas de cada ambiente.
- B) A mesma imagem pode ser usada em homologação e produção se os secrets forem mantidos fora do código. Para isso, o arquivo de User Secrets utilizado no desenvolvimento deve ser montado no container de produção e a aplicação deve carregar o mesmo identificador de secrets em todos os ambientes.
- C) As configurações não sensíveis podem permanecer em arquivos específicos de ambiente e os secrets devem ser fornecidos pelo ambiente de execução ou por um serviço seguro. A imagem deve ser única e construída em múltiplas etapas. No deploy integrado, o fallback da SPA deve ser registrado somente se app/index.html existir e deve ficar restrito a /app. No deploy separado, URLs, CORS, autenticação e contratos precisam ser coordenados.
- D) O deploy integrado pode usar a mesma imagem e servir os arquivos React. O fallback deve ser registrado para qualquer caminho não encontrado, mesmo quando app/index.html estiver ausente, pois os endpoints /api e /health mapeados anteriormente impedem que respostas HTML sejam devolvidas para caminhos desconhecidos dessas áreas.
- E) O deploy separado permite publicar front-end e back-end em ciclos independentes. Como o cliente é gerado pelo OpenAPI, essa estratégia dispensa a coordenação de URLs, CORS, cookies e versões do contrato, deixando cada pipeline responsável somente pelo artefato que produz.

Resposta correta: alternativa C.

Análise das alternativas

A) Alternativa incorreta.

Justificativa: a inclusão das diferenças de ambiente dentro da imagem produz artefatos distintos e contraria a condição de reutilizar a mesma construção. A imagem pode conter os arquivos públicos necessários, enquanto o nome do ambiente e os valores variáveis são informados durante a execução.

B) Alternativa incorreta.

Justificativa: User Secrets foi concebido para o desenvolvimento local e não representa um repositório de produção. Montar esse arquivo no container transfere para o ambiente publicado um mecanismo voltado às estações de desenvolvimento. A produção deve receber as credenciais de uma fonte administrada pelo ambiente de hospedagem.

C) Alternativa correta.

Justificativa: uma imagem imutável pode percorrer os ambientes quando os valores variáveis são fornecidos no início da execução. A construção em múltiplas etapas permite compilar com o SDK e publicar apenas os componentes necessários ao runtime. A verificação do arquivo evita registrar uma SPA inexistente e a restrição do fallback a /app impede que caminhos desconhecidos da API ou dos health checks recebam index.html. A hospedagem separada introduz comunicação entre implantações distintas.

D) Alternativa incorreta.

Justificativa: um fallback amplo pode responder com o HTML da SPA para caminhos desconhecidos, inclusive para URLs iniciadas por /api que não correspondam a um endpoint válido. Isso oculta o 404 esperado pelo cliente. Se o arquivo de entrada não existir, o mapeamento da SPA também não deve ser registrado.

E) Alternativa incorreta.

Justificativa: o cliente gerado reduz divergências na representação das operações, mas não define onde o back-end está hospedado nem configura o navegador para enviar cookies entre origens. Também não resolve sozinho a compatibilidade entre uma versão do front-end e uma nova versão da API. Esses aspectos continuam dependentes da configuração e da sequência das publicações.

REFERÊNCIAS

ALLAMARAJU, S. *RESTful web services cookbook*. Sebastopol: O'Reilly Media, 2010.

ALLEN, G.; OWENS, M. *The definitive guide to SQLite*. 2. ed. Berkeley: Apress, 2010.

BANKS, A.; PORCELLO, E. *Learning React: modern patterns for developing React apps*. 2. ed. Sebastopol: O'Reilly Media, 2020.

BARTH, A. *HTTP State Management Mechanism*. RFC Editor, 2011. Disponível em: <https://tinyurl.com/7yu7yjr>. Acesso em: 8 jul. 2026.

BRIND, M. *ASP.NET core razor pages in action*. Shelter Island: Manning Publications, 2022.

DELAMATER, M.; MURACH, J. *Murach's ASP.NET Core MVC*. 2. ed. Fresno: Mike Murach & Associates, 2022.

DE SANCTIS, V. *ASP.NET Core 8 and Angular: full-stack web development with ASP.NET Core 8 and Angular*. 6. ed. Birmingham: Packt Publishing, 2024.

FORSGREN, N.; HUMBLE, J.; KIM, G. *Accelerate: the science of lean software and DevOps: building and scaling high performing technology organizations*. Portland: IT Revolution Press, 2018.

FREEMAN, A. *Pro ASP.NET Core 7*. 10. ed. Shelter Island: Manning, 2023.

FREEMAN, A.; RATTZ, J. C. *Pro LINQ: Language Integrated Query in C#* 2010. Berkeley: Apress, 2010.

GAMMELGAARD, C. H. *Microservices in .NET*. 2. ed. Shelter Island: Manning, 2019.

GEEWAX, J. J. *API Design patterns*. Shelter Island: Manning Publications, 2021.

GOURLEY, D. *et al. HTTP: the definitive guide*. Sebastopol: O'Reilly Media, 2002.

HUMBLE, J.; FARLEY, D. *Continuous delivery: reliable software releases through build, test, and deployment automation*. Boston: Addison-Wesley, 2010.

HUNTER, K. L. *Irresistible APIs: designing web APIs that developers will love*. Shelter Island: Manning, 2016.

JENSEN, E. E. *SQLite and SQL Server Compact Toolbox*. *GitHub*, [s.d.]. Disponível em: <https://tinyurl.com/4xbzeacp>. Acesso em: 8 jul. 2026.

JONES, M. B.; BRADLEY, J.; SAKIMURA, N. *JSON Web Token (JWT)*. RFC Editor, 2015. RFC 7519. Disponível em: <https://tinyurl.com/3zfbe9uw>. Acesso em: 8 jul. 2026.

KLEPPMANN, M. *Designing Data-Intensive Applications*. Sebastopol: O'Reilly Media, 2017.



Handwriting practice lines consisting of 30 horizontal blue lines. Each line is preceded by a small blue dot, serving as a guide for letter height and placement.



Handwriting practice lines consisting of 30 horizontal blue lines. Each line is preceded by a small blue dot, serving as a starting point for letter formation. The lines are evenly spaced and extend across the width of the page.



Lined writing area with horizontal ruling lines.



Informações:
www.sepi.unip.br ou 0800 010 9000